



南京理工大学

5 贪心策略

Greedy Strategy



引例

- 假设有4种硬币，面值分别为2角5分、1角、5分和1分。现在要求以最少的硬币个数找给顾客6角3分。

基于贪心策略：面值越大，需要的个数越少。通常的做法是：先拿两个2角5分+1个1角+3个1分。

贪心策略总是做出在**当前**看来**最好的选择**，并且不从整体上考虑最优问题，所做出的每一步选择只是**局部意义上**最优选择(因而效率往往较高)，逐步扩大解的规模。当然，希望贪心策略得到的最终结果也是整体最优的(上面的解法得到的恰好是最优解)，但不能保证必定得到最优解。



例如：面值有1角1分、5分和1分，要找给顾客1角5分。如果还是使用上述贪心策略，得到的解是：1个1角1分 + 4个1分。

而实际上最优解是3个5分。

虽然贪心策略不能保证对所有问题都能得到最优解，但是对很多问题(包括很多著名的问题)都能产生整体最优解。例如，我们今天要讲的图的单元最短路径问题、最小生成树问题。在有些情况下，虽然不能得到整体最优解，但是结果却是最优解的很好近似。

使用贪心策略的难点在于：证明所设计的算法确实能够正确解决所求解的问题。(正确性证明)



Huffman树

- ◆ **结点的路径长度**定义为：
从根结点到该结点的路径上
分支的数目。（等于结点深度）
- ◆ **树的路径长度**定义为：
树中每个结点的路径长度之和。



◆ **树的带权路径长度**定义为：

树中所有叶子结点的带权路径长度之和

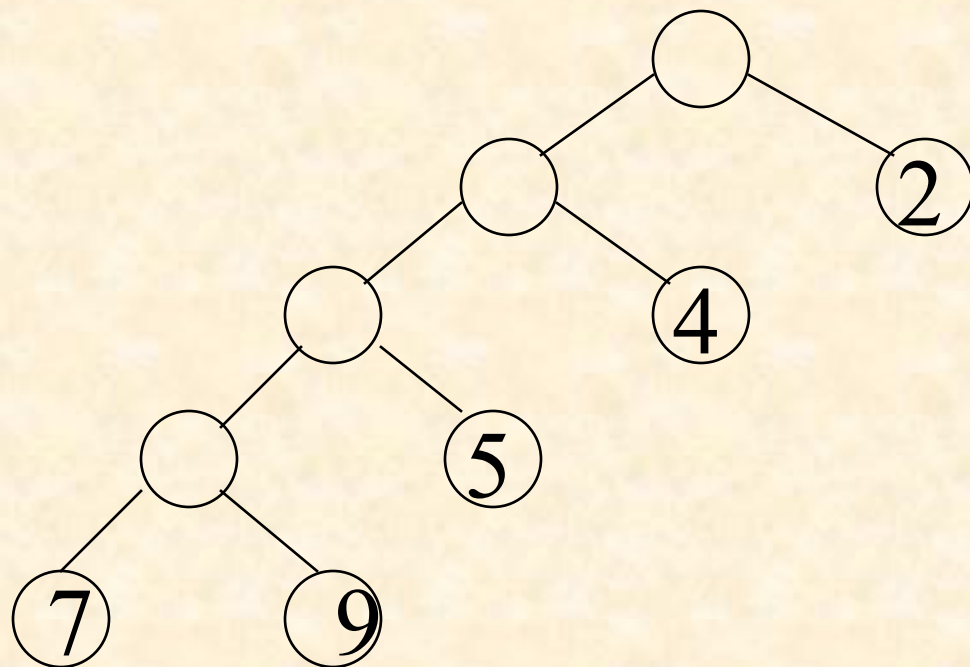
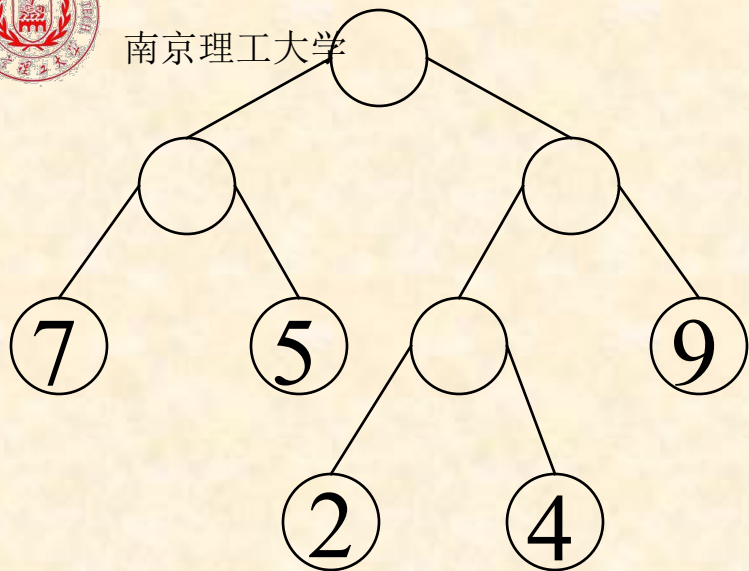
$$WPL(T) = \sum w_k l_k \text{ (对所有叶子结点)}。$$

例如：

在所有含 n 个叶子结点、并带相同权值的 m 叉树中，必存在一棵其**带权路径长度取最小值**的树，称为“**最优树**”。



南京理工大学



$WPL(T)=$

$7 \times 2 + 5 \times 2 + 2 \times 3 +$

$4 \times 3 + 9 \times 2$

$=60$

$WPL(T)=$

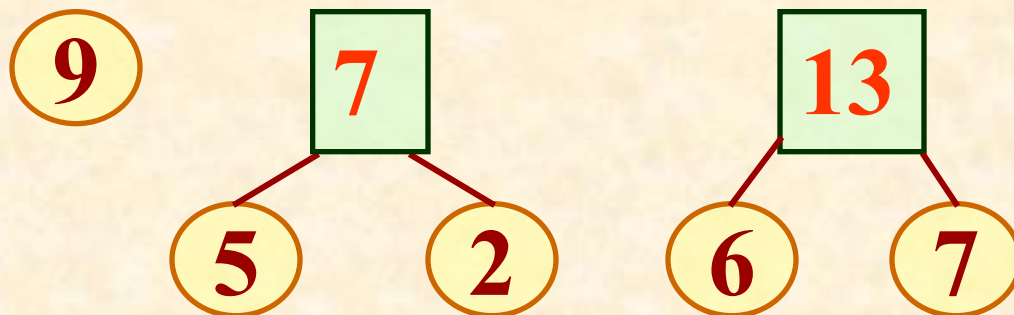
$7 \times 4 + 9 \times 4 + 5 \times 3 +$

$4 \times 2 + 2 \times 1$

$=89$

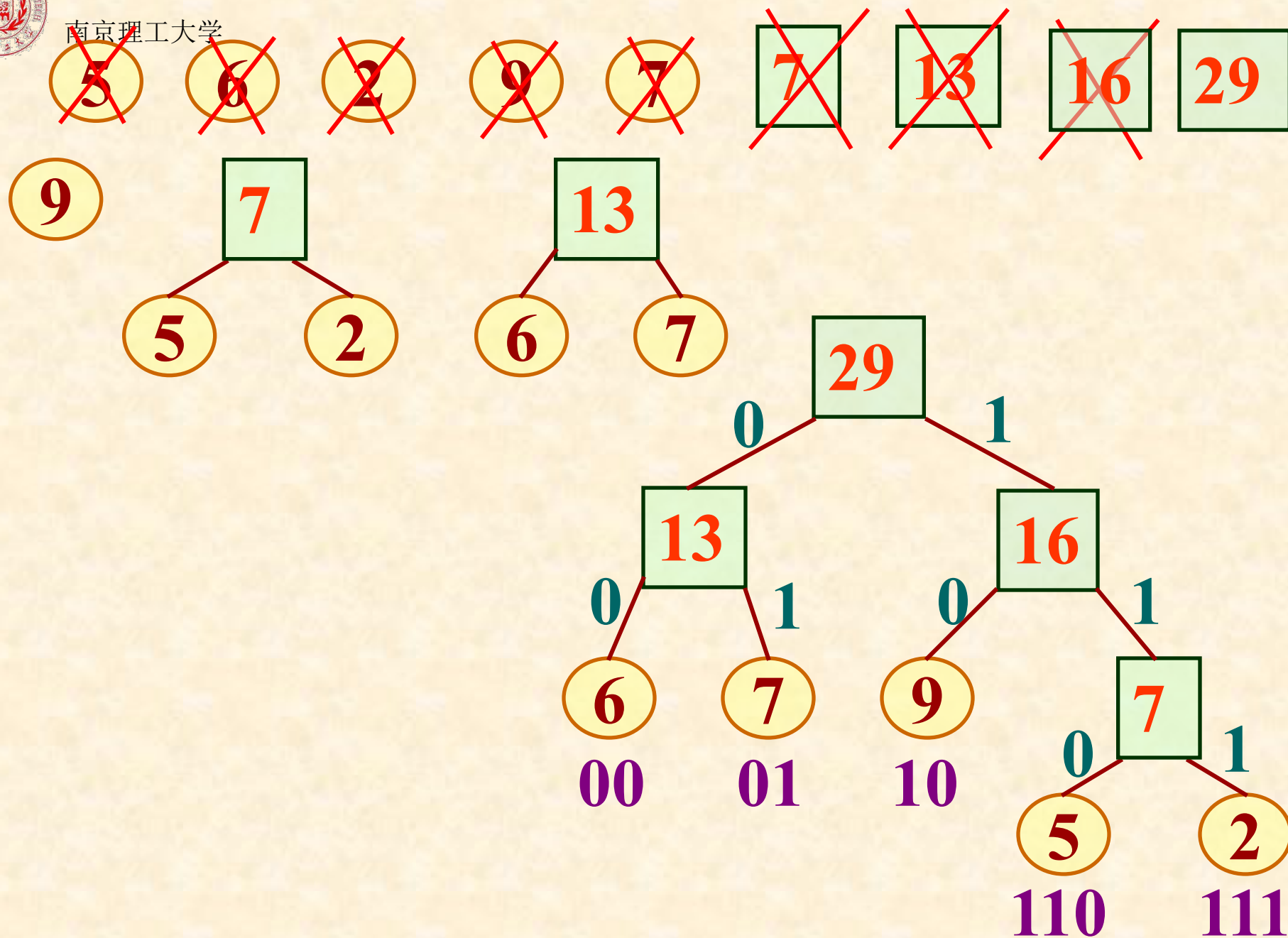


例如: 已知权值 $W=\{ 5, 6, 2, 9, 7 \}$





南京理工大学





Huffman树的数学原理—排序不等式

设有两组数 $a_1, a_2, \dots, a_n$ 和 $b_1, b_2, \dots, b_n$, 满足

$$a_1 \leq a_2 \leq \dots \leq a_n \text{ 和 } b_1 \leq b_2 \leq \dots \leq b_n$$

则有

$$\begin{aligned} a_1 b_n + a_2 b_{n-1} + \dots + a_n b_1 &\leq a_1 b_{t_1} + a_2 b_{t_2} + \dots + a_n b_{t_n} \\ &\leq a_1 b_1 + a_2 b_2 + \dots + a_n b_n \end{aligned}$$

式中 $t_1, t_2, \dots, t_n$ 是 $1, 2, \dots, n$ 的任意一个排列。

当且仅当 $a_1 = a_2 = \dots = a_n$ 或 $b_1 = b_2 = \dots = b_n$ 时, 等号成立。

以上排序不等式也可简记为:

$$\text{反序和} \leq \text{乱序和} \leq \text{同序和}$$



证明：（逐步调整法）

例如，其余不变时，将 $a_1b_1 + a_2b_2$ 调整为 $a_1b_2 + a_2b_1$ ，易知

$$(a_1b_1 + a_2b_2) - (a_1b_2 + a_2b_1) = (a_1 - a_2)(b_1 - b_2) \geq 0$$

即 $a_1b_2 + a_2b_1 \leq a_1b_1 + a_2b_2$ ；即 $n=2$ 时不等式成立。

$n=2$ 时，结论退化为：反序和 $\leq$ 同序和

采用逐步调整法，排序不等式可得证。

当 $n>2$ 时，只需分别证明两个不等式即可。

A. 乱序和 $\leq$ 同序和

B. 反序和 $\leq$ 乱序和



A. 乱序和 $\leq$ 同序和

考察 $a_1b_{t_1} + a_2b_{t_2} + \dots + a_nb_{t_n}$

若 $t_1=1$ ，那么考察 t_2 。如果 $t_i=i$ ， $i=1, \dots, k$ ，那么考察 t_{k+1} 。

现不妨设第一个满足 $t_k > k$ 的项脚标为 m （满足 $t_m > m$ ），即

$$a_1b_1 + a_2b_2 + \dots + a_{m-1}b_{m-1} + a_mb_{t_m} + \dots + a_nb_{t_n},$$

其中， $t_m > m$ ，于是： $b_{t_m} \geq b_m$ ，即 $b_m \leq b_{t_m}$

我们在后面的 $a_mb_{t_m} + \dots + a_nb_{t_n}$ 中找到含 b_m 的项，设为 a_jb_m ，

$$\text{形式为： } a_mb_{t_m} + \dots + a_jb_m + \dots + a_nb_{t_n}$$

其中， $j > m$ ，有 $a_j \geq a_m$ ，即 $a_m \leq a_j$

于是： $a_mb_{t_m} + a_jb_m$ （反序和） $\leq a_mb_m + a_jb_{t_m}$ （同序和）



已知 $a_m b_{t_m} + a_j b_m$ (反序和) $\leq a_m b_m + a_j b_{t_m}$ (同序和)

则有

$$\begin{aligned} a_1 b_1 + a_2 b_2 + \dots + a_{m-1} b_{m-1} + a_m b_{t_m} + \dots + a_j b_m + \dots + a_n b_{t_n} &\leq \\ a_1 b_1 + a_2 b_2 + \dots + a_{m-1} b_{m-1} + a_m b_m + \dots + a_j b_{t_m} + \dots + a_n b_{t_n} \end{aligned}$$

因为乱序的项是有限的，经过有限步调整，即可得同序和，
因此，乱序和 $\leq$ 同序和，证毕。

B. 反序和 $\leq$ 乱序和

与 A 的证明完全相似，每步进行缩小后经有限步即可证明。



排序不等式——一个例子

- 例：有**10**人各拿一只水桶去接水，水龙头只有一个，灌满这些水桶需要 a_i 分钟。假设 a_i 大小不同，不妨设 $a_1 < a_2 < \dots < a_{10}$ 。问：应该如何安排这**10**人的顺序，使得他们等待的总时间最少？
- 解：最短时间 $a_1 * 10 + a_2 * 9 + \dots + a_{10} * 1$ （反序和）
最长时间 $a_1 * 1 + a_2 * 2 + \dots + a_{10} * 10$ （同序和）



Huffman树构造算法—从数学角度分析

- 贪心性质
 - ✓ 当前要合并的总是权值（频度）最小的两个结点；
 - ✓ 先考虑的结点深度比较大，对应的编码比较长。
- 与排序不等式的关系：

频度小的编码长，频度大的编码短（即反序和，为最小），这样编码后，总编码长度最小。
- **工程应用问题→图论问题→纯粹数学问题**



Huffman树的算法实现:

- 由于总是在 $C=\{c_1, \dots, c_n\}$ 中寻找最小频度对应的节点 c 和 c' ,
- 删除 c 和 c' , 生成新节点 v (其频度 $= c+c'$), 再将 v 插入集合 C , (节点数减1),
- 再寻找最小频度对应的节点;
- 直到最后只有一个节点时, **Huffman**树构造完成。
- (上述过程重复 $n-1$ 次)
- 由于需要不断选择最小节点, 插入新节点, 因此, 最小堆结构是最佳选择。



Huffman树算法及时间复杂度分析

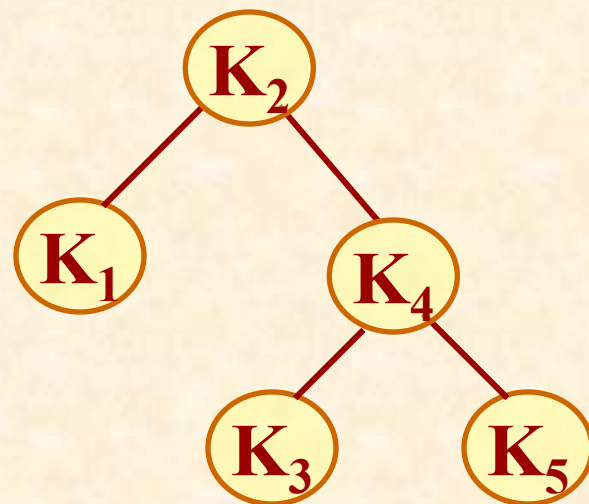
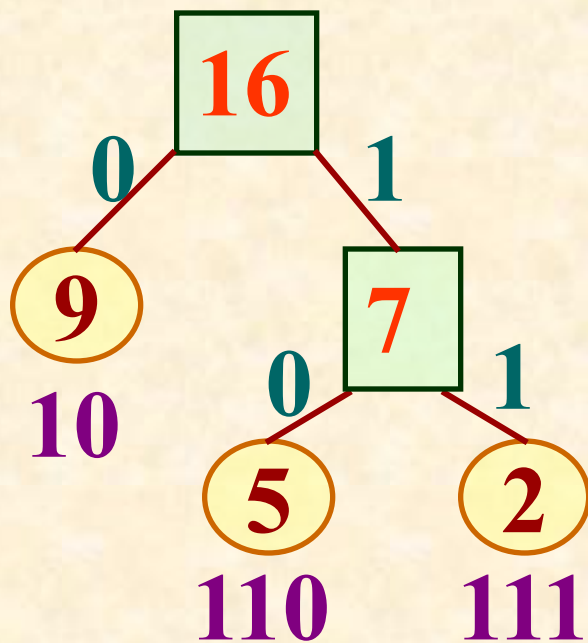
输入：n个字符的集合 $C=\{c_1, \dots, c_n\}$ 及其频度 $=\{f_1, \dots, f_n\}$

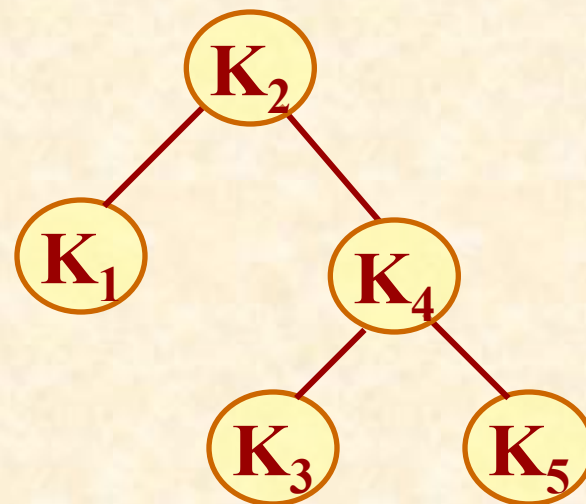
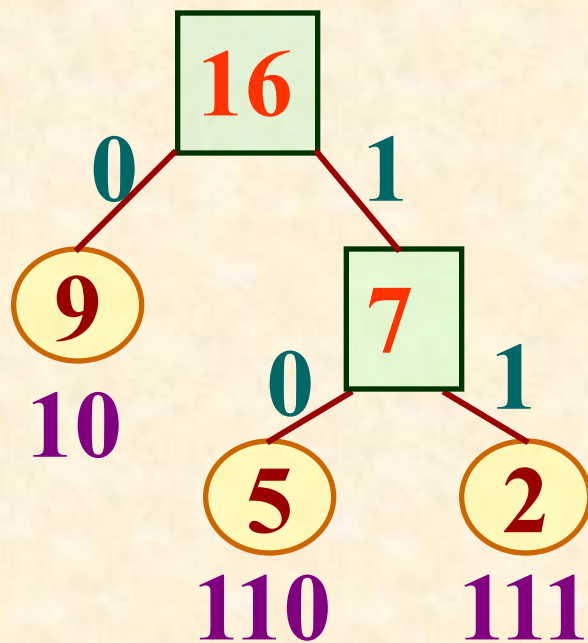
输出：C的Huffman树

1. 根据频度，将所有字符建立最小堆 H $// \Theta(n)$
2. $V \leftarrow C; T = \{\}$
3. for $j \leftarrow 1$ to $n-1$ $// n$ 个节点经 $n-1$ 次合并，每次少1个节点
4. $c \leftarrow \text{Delete_Min}(H)$ $// \lfloor \log(n-1) \rfloor + \lfloor \log(n-2) \rfloor + \dots + \lfloor \log 2 \rfloor + \lfloor \log 1 \rfloor = \Theta(n \log n)$
5. $c' \leftarrow \text{Delete_Min}(H)$ $// \lfloor \log(n-2) \rfloor + \lfloor \log(n-3) \rfloor + \dots + \lfloor \log 1 \rfloor = \Theta(n \log n)$
6. $f(v) \leftarrow f(c) + f(c')$
7. $\text{INSERT}(H, v)$ $// \lfloor \log(n-1) \rfloor + \lfloor \log(n-2) \rfloor + \dots + \lfloor \log 2 \rfloor = \Theta(n \log n)$
8. $V = V \cup \{v\}$ $// \Theta(1) * (n-1) = \Theta(n)$
9. $T = T \cup \{(v, c), (v, c')\}$
10. end while $// \text{总时间复杂度 } \Theta(n \log n)$

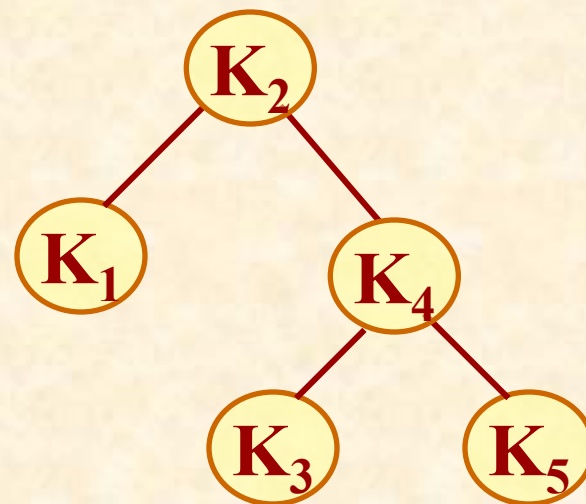
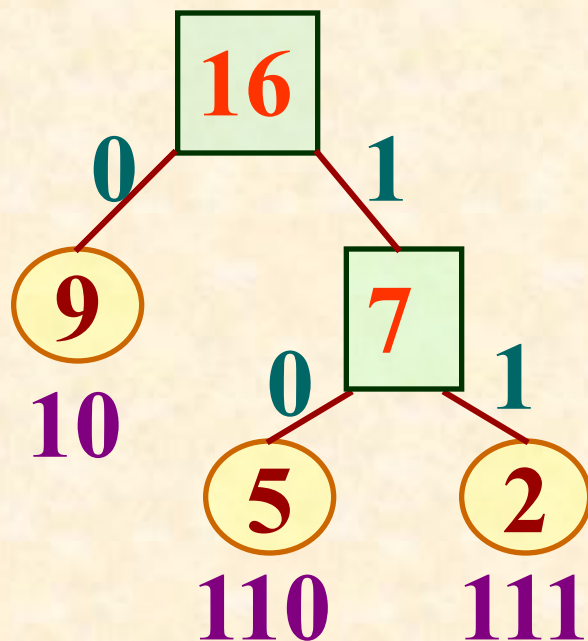


进一步的研究：Huffman树 vs. 静态最优查找树 (最优二叉查找树)

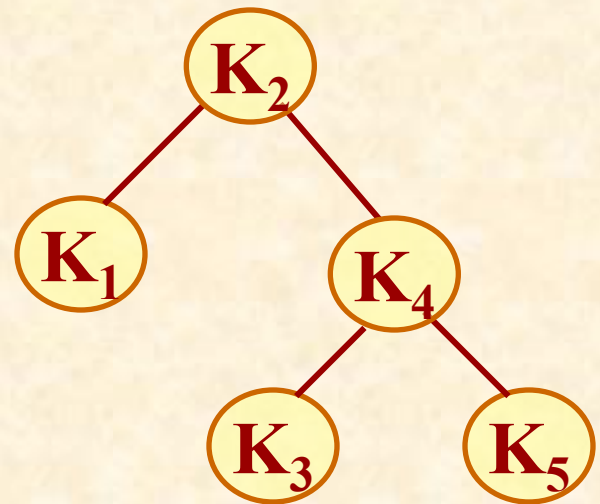
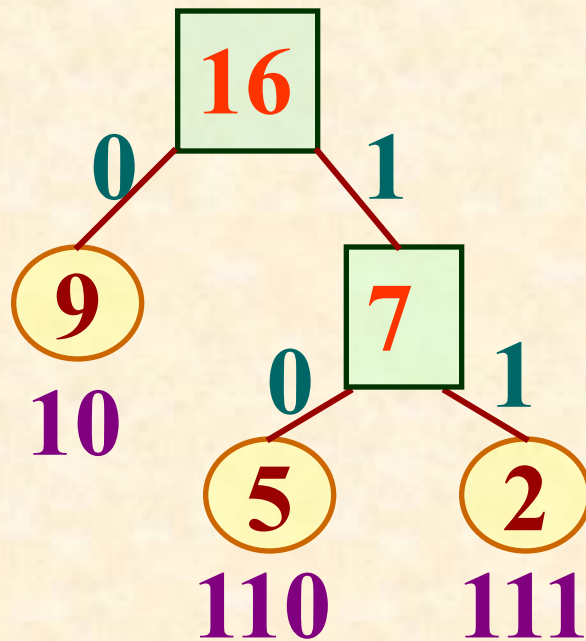




	优化目标函数 min	
	Huffman树	最优二叉查找树
根高度为0	$\sum_i p_i h_i$	$\sum_i p_i (h_i + 1) \Leftrightarrow \sum_i p_i h_i$
根高度为1	$\sum_i p_i (h_i - 1) \Leftrightarrow \sum_i p_i h_i$	$\sum_i p_i h_i$



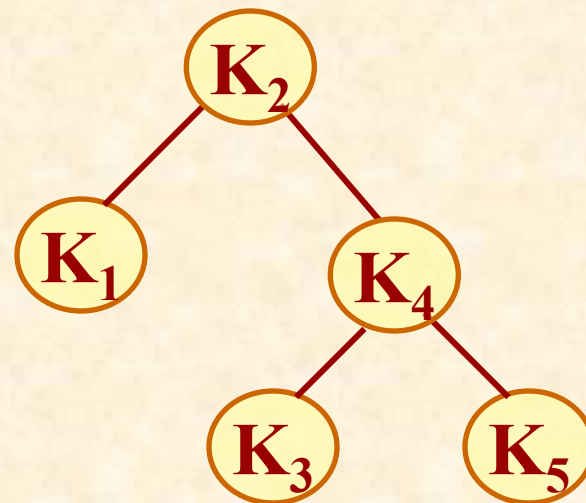
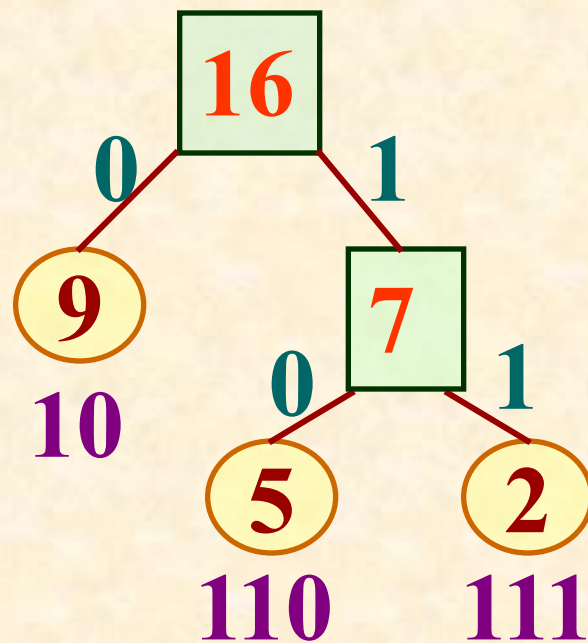
	结点限制	
	Huffman树	最优二叉查找树
实际结点	仅限于叶子	所有结点 (暂不考虑失败结点)
虚拟结点	非叶子结点 (无意义)	查找失败结点 (有权值, 有意义)



结点关键字的有序性:

最优二叉查找树的中序遍历非递减有序,

Huffman树的关键字 (即权值) 无此要求。



等价？

虽然 **目标函数相同**，但构造的两种树不可能相同，
因为 **约束条件不同**： Huffman树结点必须强制为叶子（否则不再是前缀码），最优二叉查找树的键值必须中序有序（否则不再是查找树）。



单源最短路径问题

- $G = (V, E)$ 是一个有向图，每条边上有一个非负整数表示长度值，其中有一个顶点称为源节点。所谓的单源最短路径问题就是：求解该源节点到所有其它节点的最短路径值。
- 不失一般性，假设 $V = \{1, 2, 3, \dots, n\}$ 并且源节点 $s = 1$ 。那么该问题可以使用 **Dijkstra** 算法来求解，该算法是一种贪心算法，并且能求得最优解。



狄克斯特拉(Edsger Dijkstra) 1930-2002

- 1956年，成功设计并实现了在有障碍物的两个地点之间找出一条**最短路径**的高效算法，这个算法被命名为“狄克斯特拉算法”，解决了机器人学中的一个十分关键的问题，即运动路径规划问题，至今仍被广泛应用。
- 1968年3月，Communications of ACM登出了狄克斯特拉的一封信，在信中他根据自己编程的实际经验和大量观察，得出如下结论：一个程序的易读性和易理解性同其中所包含的无条件转移控制的个数成反比关系，也就是说，转向语句的个数愈多，程序就愈难读、难懂。因此他认为“goto语句是有害的”，并从而启发了结构化程序设计的思想。
- 1972年的图灵奖授予荷兰的计算机科学家埃德斯加.狄克斯特拉。
- 1983年，ACM为纪念Communications of ACM 创刊25周年，评选出从1958-1982的四分之一的一个世纪中在该杂志上发表的25篇有里程碑意义的论文，每年一篇，狄克斯特拉一人就有两篇入选，是仅有的这样的两位学者之一。
- 在程序设计技术、算法和算法理论、编译器、操作系统等诸多方面，狄克斯特拉都有许多创造，作出了杰出贡献。



- 开始时，我们将所有的节点划分为两个集合 $X = \{1\}$, $Y = \{2, 3, 4, \dots, n\}$ 。所有已经计算好的节点存放在 X 中， Y 中表示还没有计算好的。 Y 中的每个节点 y 有一个对应的量 $\lambda[y]$ ，该值是从源节点到 y (并且只经由 X 中的节点) 的最短路径值。
- Dijkstra算法
 - 假设 $V = \{1, 2, 3, \dots, n\}$ 并且 $s = 1$ 。
 - 选择一个 $\lambda[y]$ 最小顶点 $y \in Y$ ，并将其移动到 X 中。
 - 若 y 被从 Y 移动到 X 中， Y 中每个和 y 相邻的顶点 w 的 $\lambda[w]$ 都要更新(表示经由 y 到 w 的一条更短的路径被发现)。



算法框架:

1. $X \leftarrow \{1\}; Y \leftarrow V - \{1\}$
2. 对于任意一个 $v \in V$, 如果存在一条边从 1 到 v , 那么 $\lambda[v] = \text{该边的长度}$ 。否则 $\lambda[v] = \infty$; 并设定 $\lambda[1] = 0$ 。

3. while $Y \neq \Phi$

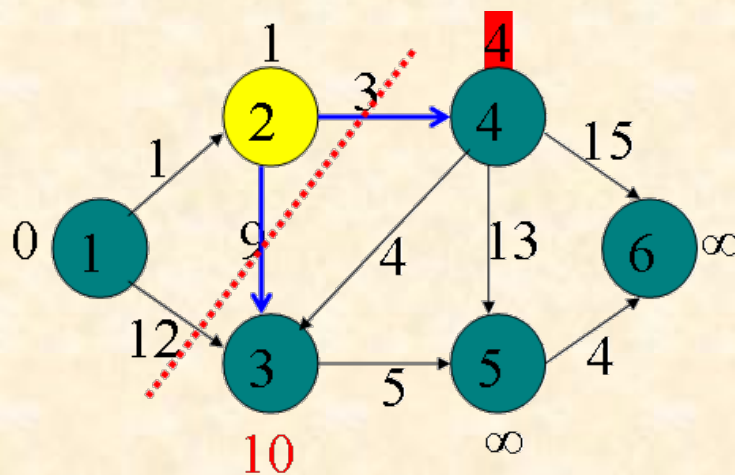
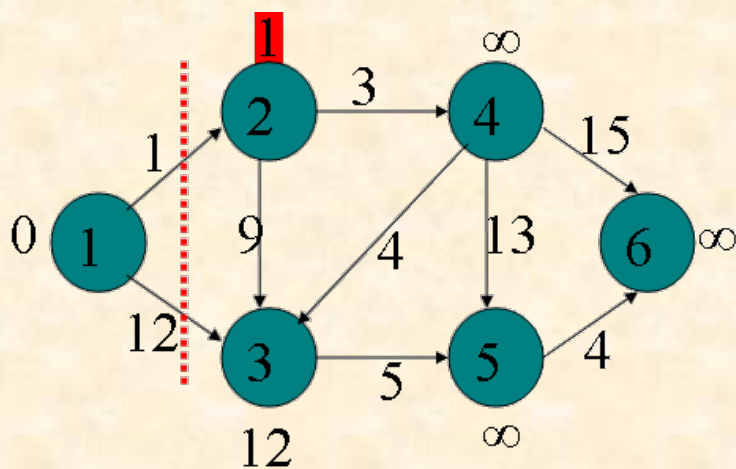
对于 $y \in Y$, 找到最小 $\lambda[y]$

将 y 从 Y 移动到 X 中

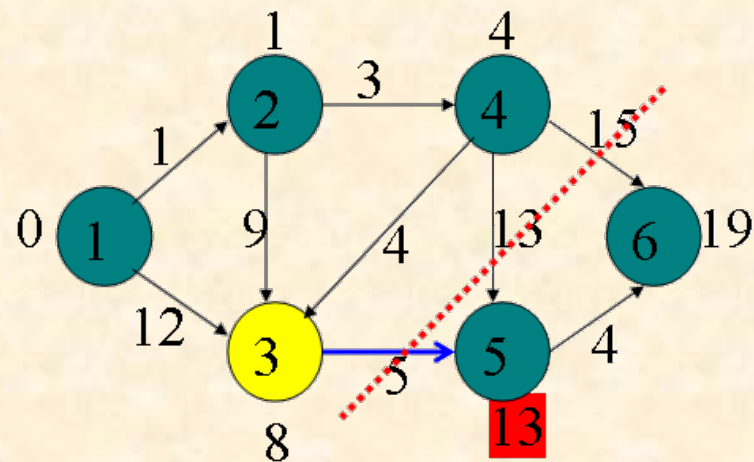
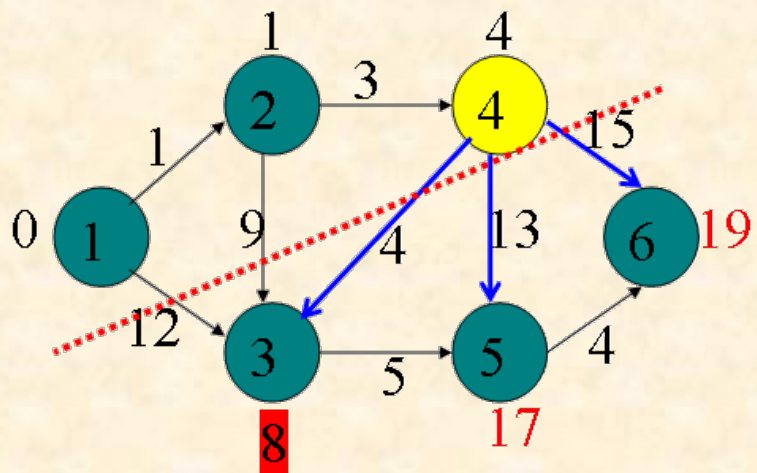
更新 Y 中和 y 相邻顶点的 λ 值

end while

if $w \in Y$ and $(y, w) \in E$
and $\lambda[y] + \text{length}[y, w] < \lambda[w]$
then $\lambda[w] \leftarrow \lambda[y] + \text{length}[y, w]$

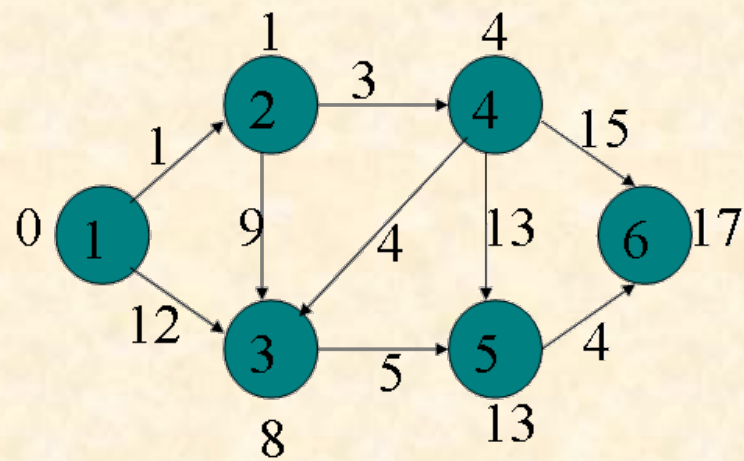
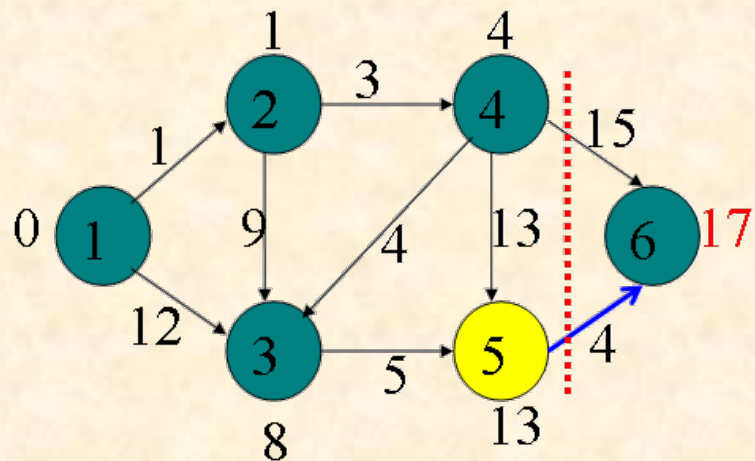


顶点i	1	2	3	4	5	6	比较次数
$\lambda[i]$		1✓	12	∞	∞	∞	n-2
			↓10	↓4✓	∞	∞	n-3



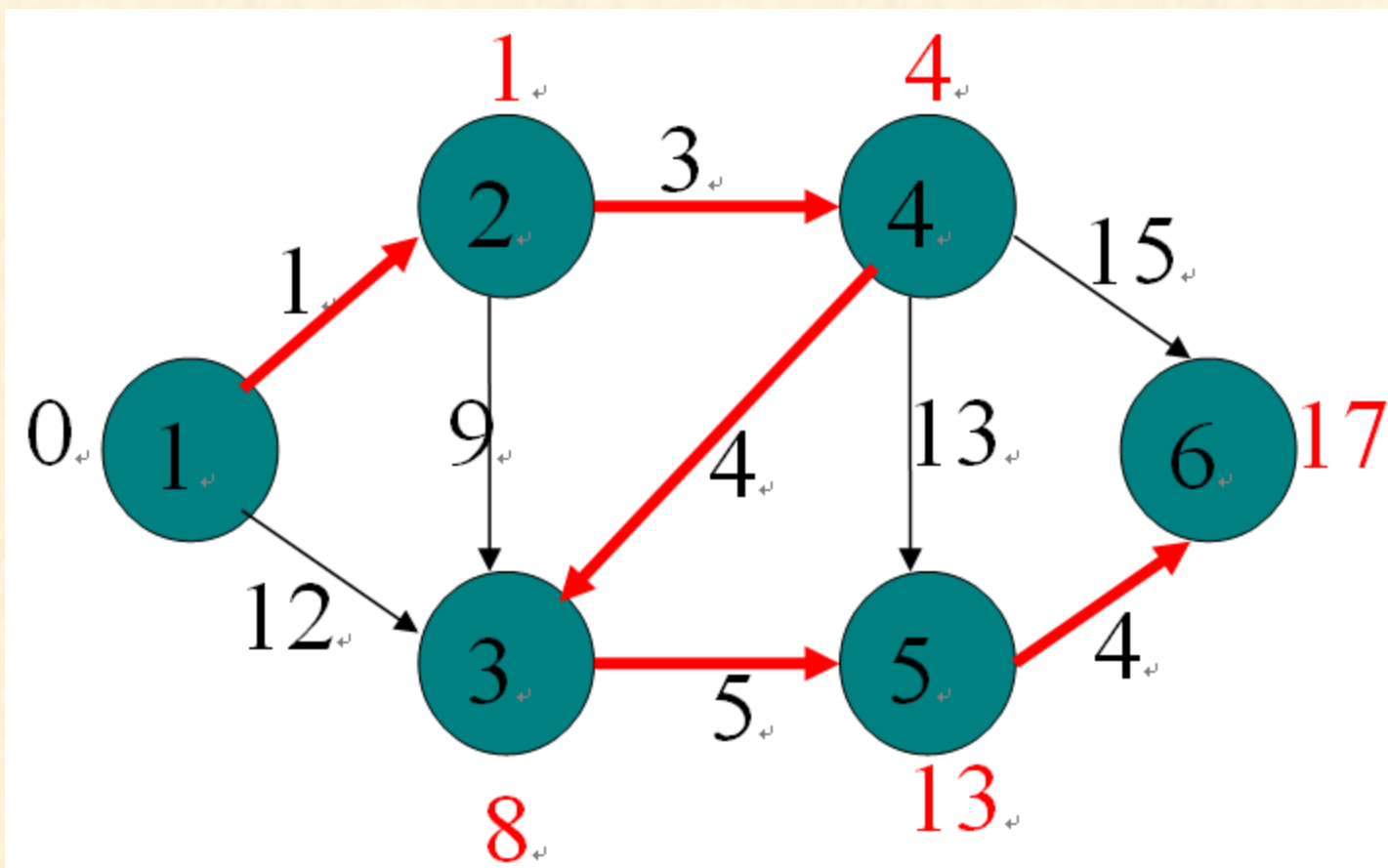
顶点i	1	2	3	4	5	6
$\lambda[i]$		1✓	12	∞	∞	∞
$\lambda[i]$			↓10	↓4✓	∞	∞
$\lambda[i]$			↓8✓		↓17	↓19
$\lambda[i]$					↓13✓	19

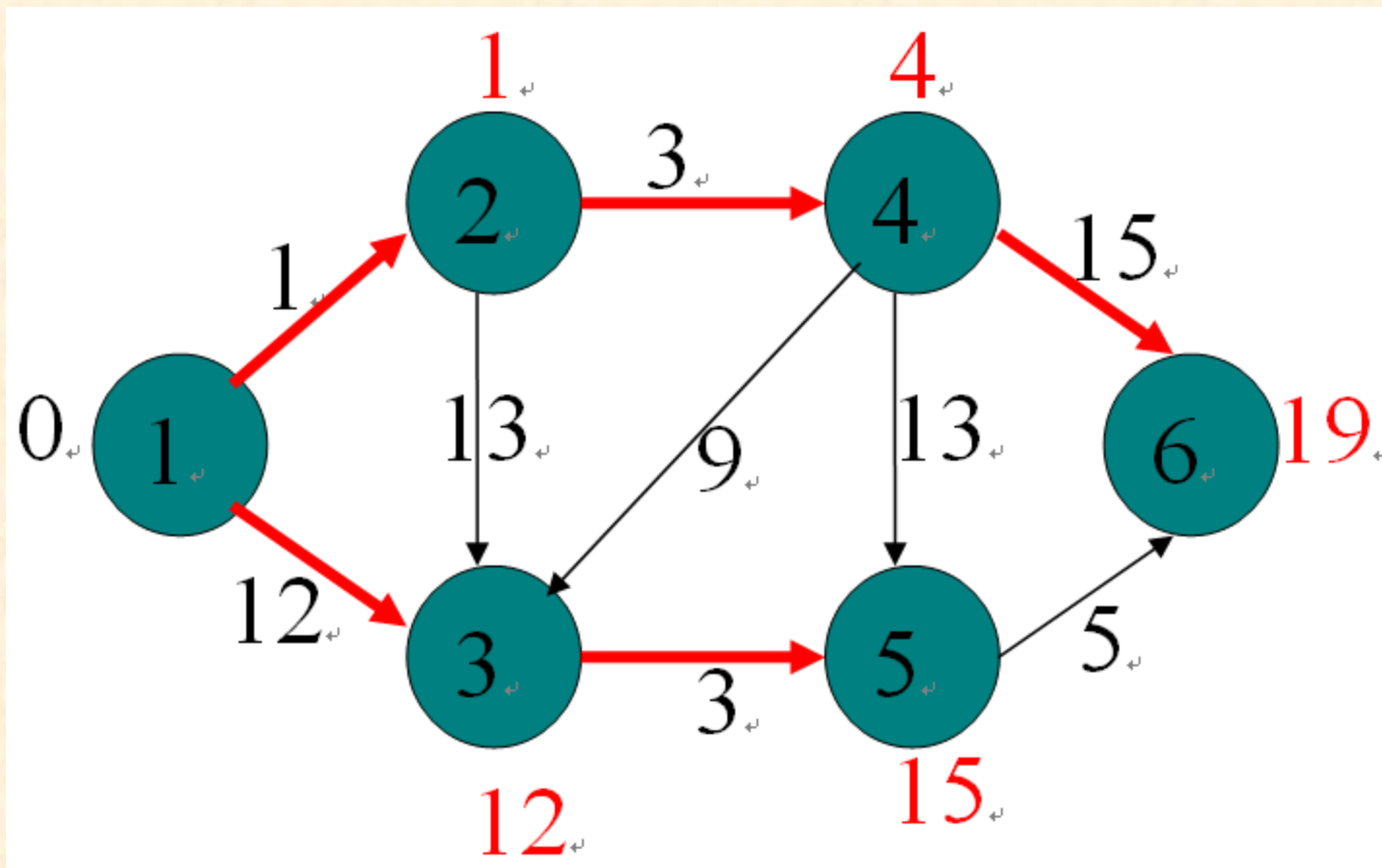
比较次数
n-2
n-3
n-4
n-5

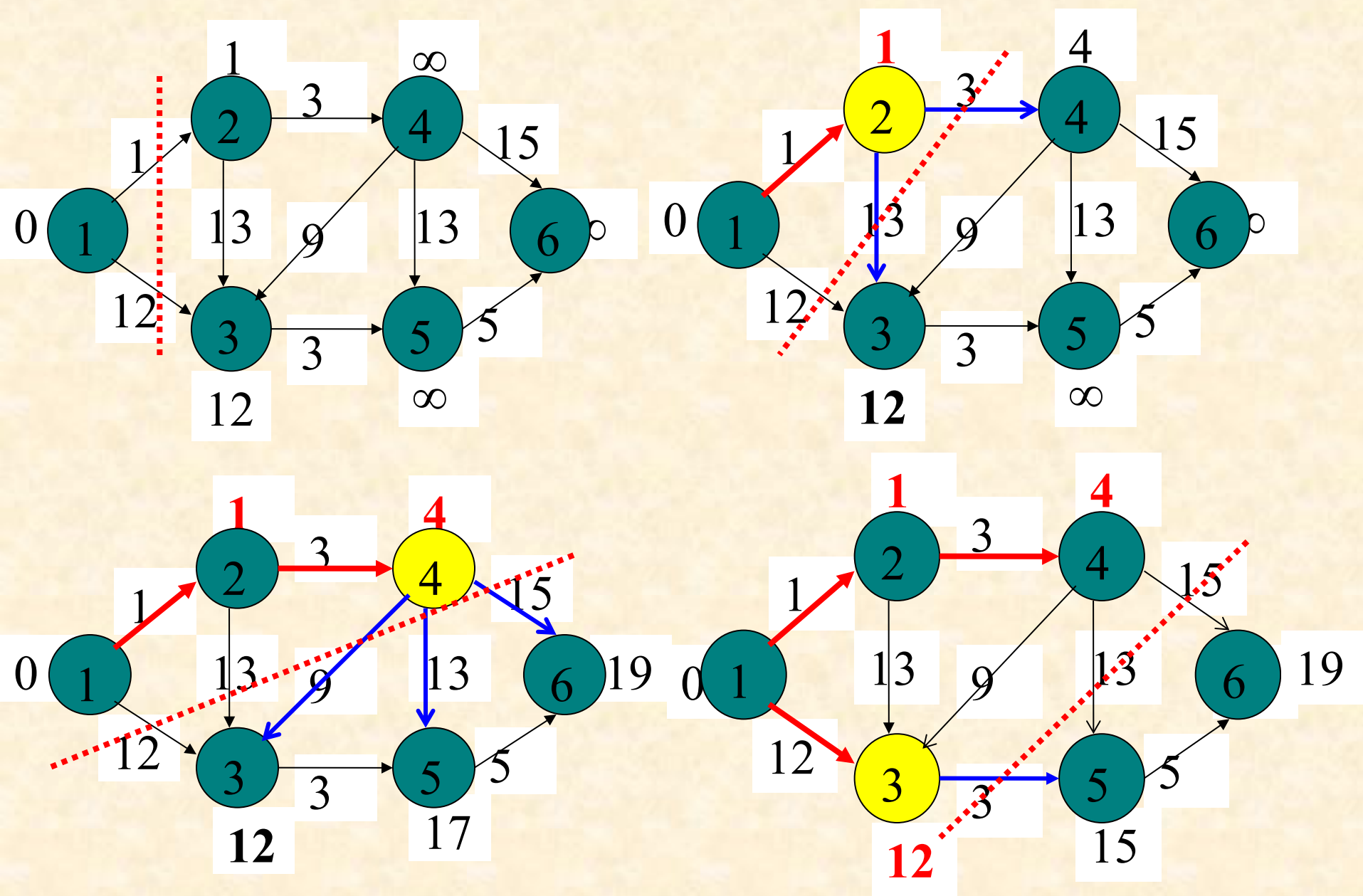


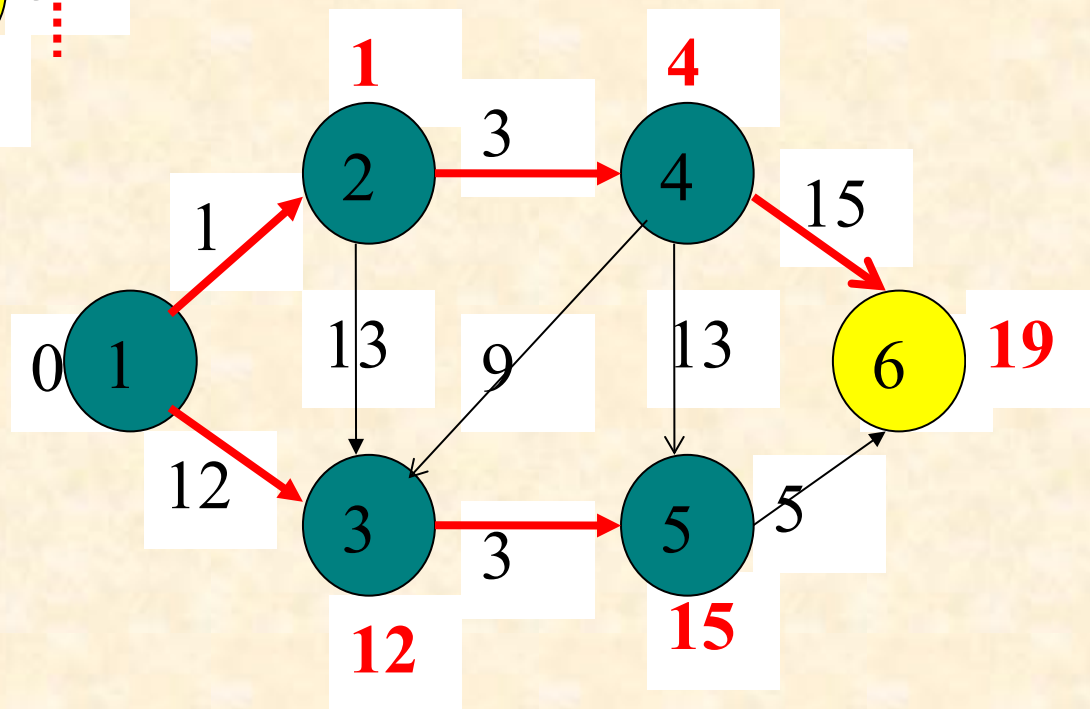
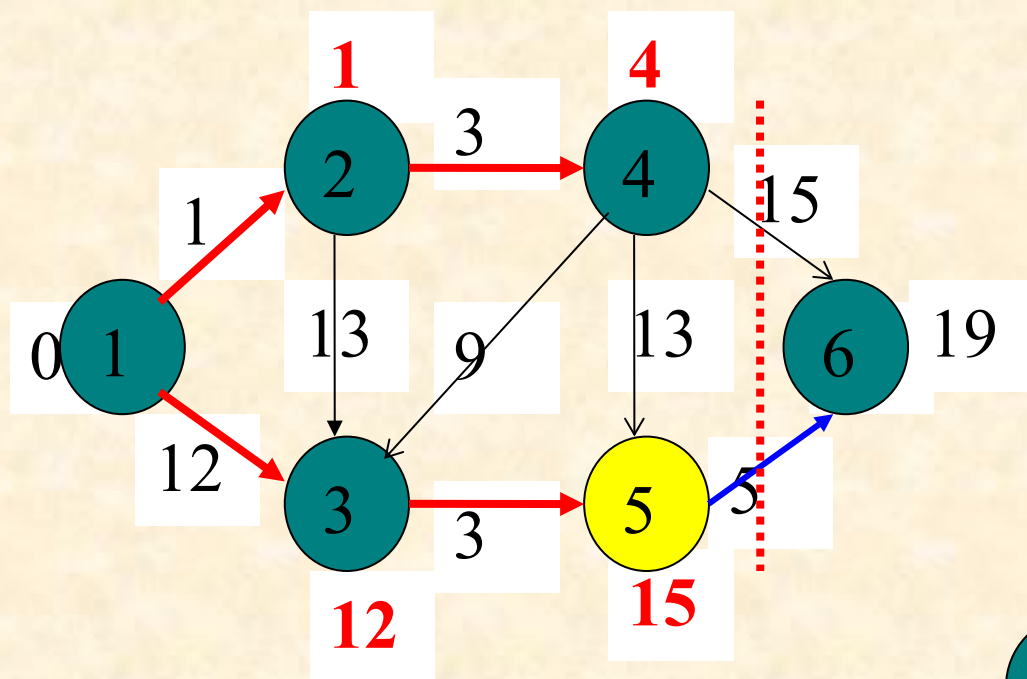
顶点i	1	2	3	4	5	6
$\lambda[i]$		1✓	12	∞	∞	∞
$\lambda[i]$			↓10	↓4✓	∞	∞
$\lambda[i]$			↓8✓		↓17	↓19
$\lambda[i]$					↓13✓	19
$\lambda[i]$						↓17✓
$\lambda[i]$		1✓	8✓	4✓	13✓	17✓

比较次数
n-2
n-3
n-4
n-5=1
0
$\sum(n-i)\{i=2\sim n\} = \Theta(n^2)$







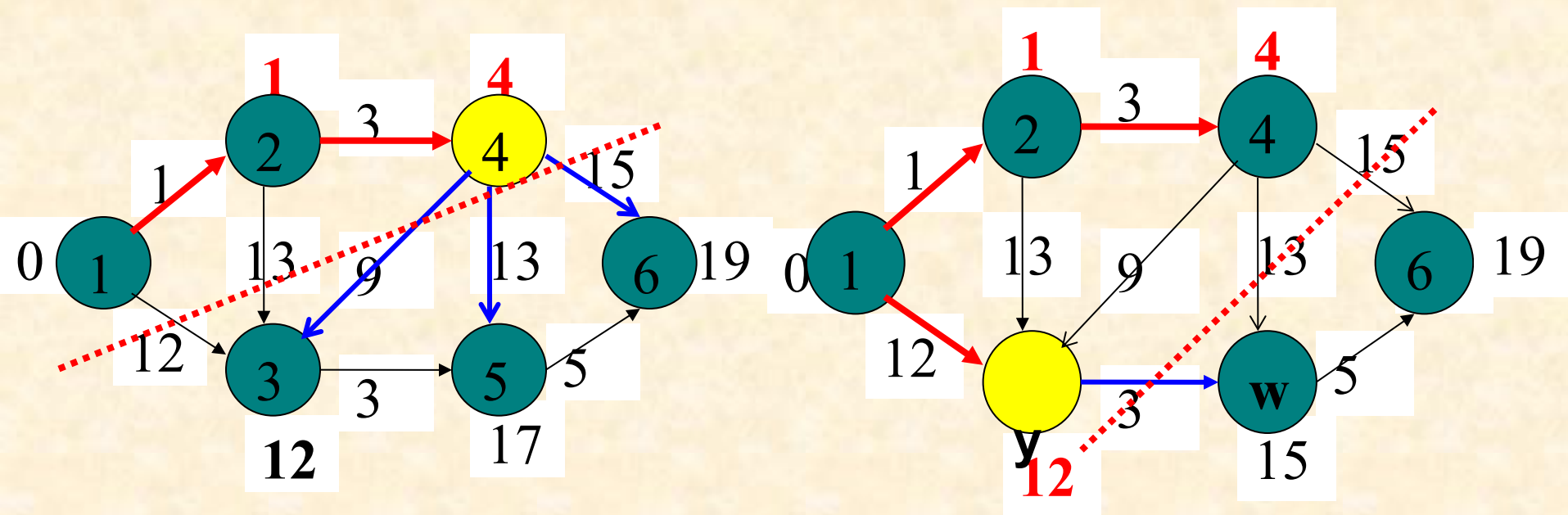




算法框架：

1. $X \leftarrow \{1\}; Y \leftarrow V - \{1\}$
2. 对于任意一个 $v \in V$ ，如果存在一条边从 1 到 v ，
那么 $\lambda[v] = \text{该边的长度}$ 。否则 $\lambda[v] = \infty$ ；并设定 $\lambda[1] = 0$ 。
3. while $Y \neq \Phi$
 对于 $y \in Y$ ，找到最小 $\lambda[y]$
 将 y 从 Y 移动到 X 中
 更新 Y 中和 y 相邻顶点的 λ 值
end while

if $w \in Y$ and $(y, w) \in E$
and $\lambda[y] + \text{length}[y, w] < \lambda[w]$
then $\lambda[w] \leftarrow \lambda[y] + \text{length}[y, w]$
 $\text{pre}[w] \leftarrow y$





算法结束后，每个节点的 λ 值就是从源节点到该节点的最短路径值。

回忆：Y中的每个节点y有一个对应的量 $\lambda[y]$ ，该值是从源节点到y
(并且只经由X中的节点)的最短路径值。

正确性证明：对于任意一个节点 $v \in V$, 使用 $\delta[v]$ 表示源节点到节点v的真正最短路径值，下面我们证明, Dijkstra算法结束后有 $\lambda[v] = \delta[v]$ 。

证明: (数学归纳法)

- 1) 显然 $\lambda[1] = \delta[1] = 0$ 。
- 2) 假设当前将 y 从 Y 中 移动到 X 中，并且在 y 之前移动到 X 中的任何一个顶点c，都有 $\lambda[c] = \delta[c]$ 。（第二数学归纳法）
- 3) 下面证明 $\lambda[y] = \delta[y]$ 。



我们知道：必定存在一条从源节点1 到节点y的**真正最短路径**，该路径长度值用 $\delta[y]$ 表示，并且这条最短路径总可以用以下节点序列表示：

$1 \rightarrow [] \rightarrow [] \rightarrow \dots \rightarrow [] \rightarrow [] \rightarrow [] \rightarrow y$



沿此方向(逆向)找到第一个属于X的节点，不妨称之为x。

分析：[]中的节点，不属于X，就属于Y，必居其一。

$1 \rightarrow [] \rightarrow [] \rightarrow \dots \rightarrow [] \rightarrow [x] \rightarrow y$ (A)

$1 \rightarrow [] \rightarrow [] \rightarrow \dots \rightarrow [x] \rightarrow [w] \dots \rightarrow y$ (B)

1个或1个以上属于Y的节点。



$$1 \rightarrow [\] \rightarrow [\] \rightarrow \dots \rightarrow [\] \rightarrow [x] \rightarrow y \quad (A)$$

$\lambda[y] \leq \lambda[x] + \text{length}[x, y]$ //算法要求

//选中x的时候，必然要更新邻接顶点 $\lambda[y]$,

//若 $\lambda[y] < \lambda[x] + \text{length}[x, y]$, 无需更新, 不等式成立;

//若 $\lambda[y] > \lambda[x] + \text{length}[x, y]$, 则 $\lambda[y]$ 更新为

// $\lambda[y] = \lambda[x] + \text{length}[x, y]$ 。

//总之, $\lambda[y] \leq \lambda[x] + \text{length}[x, y]$ 成立。

$= \delta[x] + \text{length}[x, y]$ //归纳假设

$= \delta[y]$ //(A)是最短路径

//由最短路径的最优子结构性质决定的

于是 **$\lambda[y] = \delta[y]$**



$$1 \rightarrow [\] \rightarrow [\] \rightarrow \dots \rightarrow [x] \rightarrow [w] \dots \rightarrow y \quad (B)$$

$$\lambda[y] \leq \lambda[w] \quad // \text{由于 } y \text{ 在 } w \text{ 之前离开 } Y$$

$$\leq \lambda[x] + \text{length}[x, w] \quad // \text{算法要求, 原因同(A)}$$

//选中x的时候, 必然要更新邻接顶点 $\lambda[w]$,

//若 $\lambda[w] < \lambda[x] + \text{length}[x, w]$, 无需更新, 不等式成立;

若 $\lambda[w] > \lambda[x] + \text{length}[x, w]$, 则 $\lambda[w]$ 更新为

$$\lambda[w] = \lambda[x] + \text{length}[x, w]。$$

总之, $\lambda[w] \leq \lambda[x] + \text{length}[x, w]$ 成立。

$$= \delta[x] + \text{length}[x, w] \quad // \text{归纳假设}$$

$$= \delta[w] \quad // \text{因为(B)是最短路径, 最优子结构性质}$$

$$\leq \delta[y] \quad // \text{因为(B)是最短路径, 最优子结构性质}$$

// (B)是最短路径, w在y的前面, 所以 $\delta[w] \leq \delta[y]$ 成立

于是 $\lambda[y] = \delta[y]$



Dijkstra算法

```
1.  $X = \{1\}; Y \leftarrow V - \{1\}; \lambda[1] \leftarrow 0$ 
2. for  $y \leftarrow 2$  to  $n$ 
3.   if  $y$  相邻于  $1$  then  $\lambda[y] \leftarrow \text{length}[1, y]$  //  $\Theta(n)$ 
4.   else  $\lambda[y] \leftarrow \infty$  //  $\Theta(n)$ 
5.   end if
6. end for
7. for  $j \leftarrow 1$  to  $n-1$ 
8.   令  $y \in Y$ , 使得  $\lambda[y]$  为最小的 //  $\sum_{i=2}^n (n-i) = (n-2)(n-1)/2 = \Theta(n^2)$ 
9.    $X \leftarrow X \cup \{y\}$  //  $1 * (n-1) = \Theta(n)$ 
10.   $Y \leftarrow Y - \{y\}$  //  $1 * (n-1) = \Theta(n)$ 
11.   for 每条边  $(y, w)$  // 每条边恰好检查一次,  $\Theta(m), m = |E|$ 
12.     if  $w \in Y$  and  $\lambda[y] + \text{length}[y, w] < \lambda[w]$  then
13.        $\lambda[w] \leftarrow \lambda[y] + \text{length}[y, w]$  //  $\Theta(m)$ 
14.     end if
15.   end for
16. end for
```

算法的时间复杂度:

$$T(n) = \Theta(n) + O(n) + \Theta(n^2) + \Theta(n) + \Theta(n) + \Theta(m) + \Theta(m) = \Theta(m + n^2)$$



算法的堆优化

- 动机: $\Theta(n^2) \rightarrow \Theta(n \log n)$?

Dijkstra算法

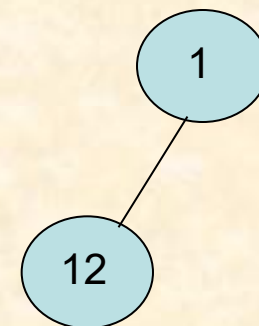
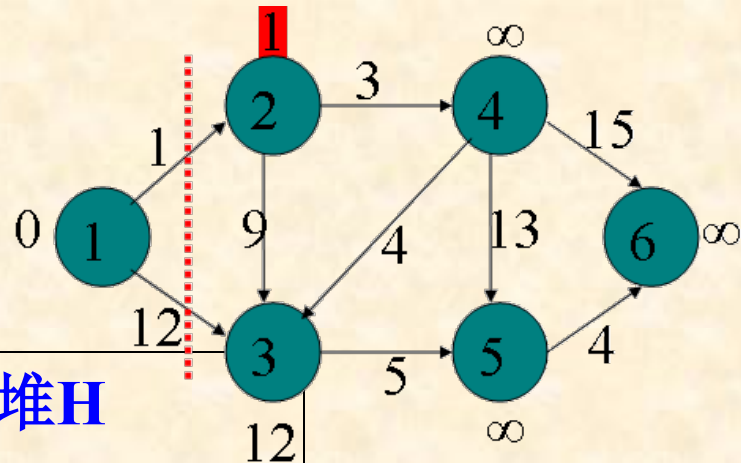
```
1.  $X = \{1\}; Y \leftarrow V - \{1\}; \lambda[1] \leftarrow 0$ 
2. for  $y \leftarrow 2$  to  $n$ 
3.   if  $y$  相邻于  $1$  then  $\lambda[y] \leftarrow \text{length}[1, y]$  //  $\Theta(n)$ 
4.   else  $\lambda[y] \leftarrow \infty$  //  $\Theta(n)$ 
5.   end if
6. end for
7. for  $j \leftarrow 1$  to  $n-1$ 
8.   令  $y \in Y$ , 使得  $\lambda[y]$  为最小的 //  $\sum_{i=2, \dots, n} (n-i) = \Theta(n^2)$ 
9.    $X \leftarrow X \cup \{y\}$  //  $1 * (n-1) = \Theta(n)$ 
10.   $Y \leftarrow Y - \{y\}$  //  $1 * (n-1) = \Theta(n)$ 
11.   for 每条边  $(y, w)$  // 每条边恰好检查一次,  $\Theta(m)$ ,  $m = |E|$ 
12.     if  $w \in Y$  and  $\lambda[y] + \text{length}[y, w] < \lambda[w]$  then
13.        $\lambda[w] \leftarrow \lambda[y] + \text{length}[y, w]$  //  $\Theta(m)$ 
14.     end if
15.   end for
16. end for
```



算法的堆优化：初始化

Dijkstra算法 //假设已初始化一个空堆H

1. $X = \{1\}; Y \leftarrow V - \{1\}; \lambda[1] \leftarrow 0$
2. for $y \leftarrow 2$ to n
3. if y 邻接于1 then
4. $\lambda[y] \leftarrow \text{length}[1, y]$
5. $\text{key}(y) \leftarrow \lambda[y]$
6. INSERT(H, y) //次数等于顶点1的邻接点数
7. else
8. $\lambda[y] \leftarrow \infty$
9. $\text{key}(y) \leftarrow \lambda[y]$ //次数等于顶点1的非邻接点数
10. end if
11. end for



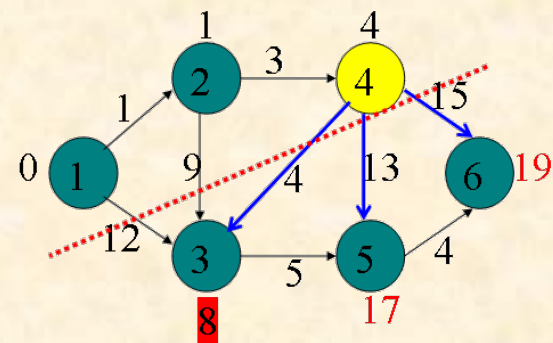
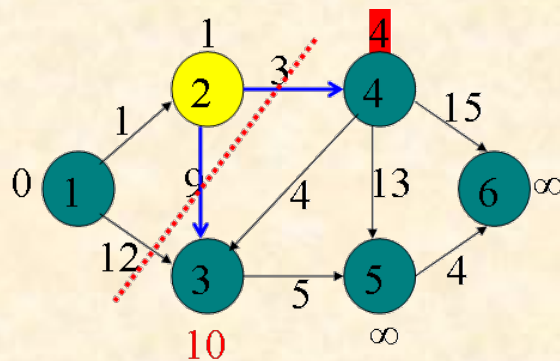
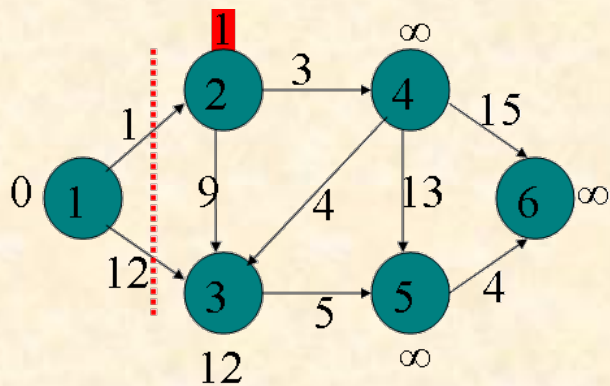


最短路径算法的堆优化：主要进程

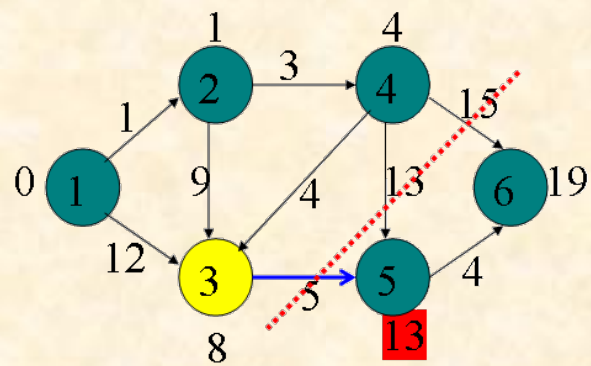
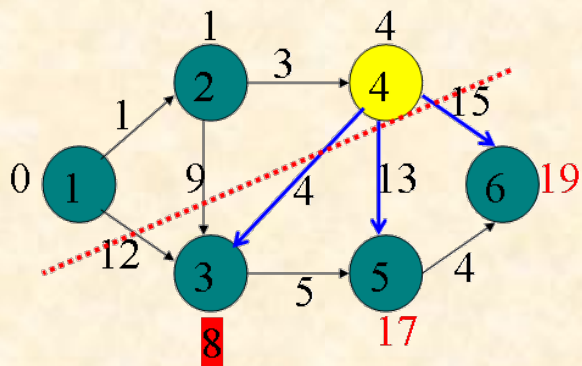
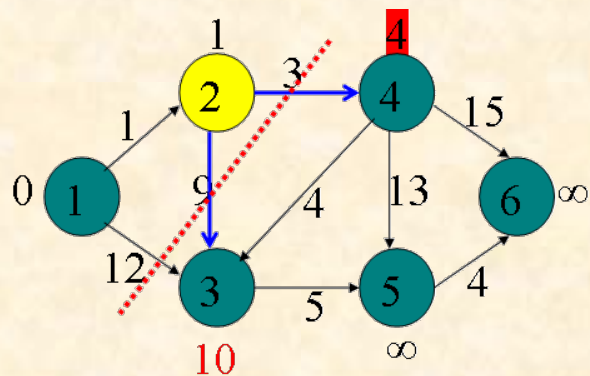
```

12. for j ← 1 to n-1
13.   y ← DELETEMIN(H) //  $\lambda[y]$  最小的顶点被挑中
14.    $Y \leftarrow Y - \{y\}, X \leftarrow X \cup \{y\}$ ,
15.   for 每个邻接于 y 的顶点  $w \in Y$ 
16.     if  $\lambda[y] + \text{length}[y, w] < \lambda[w]$  then
17.        $\lambda[w] \leftarrow \lambda[y] + \text{length}[y, w]$ 
18.        $\text{key}(w) \leftarrow \lambda[w]$ 
19.     end if
20.     if  $w \notin H$  then INSERT(H, w) //  $\lambda[y]$  从  $\infty$  降为有限大, 入堆参与竞选
21.     else SIFT-UP(H,  $H^{-1}(w)$ ) //  $\lambda[y]$  从有限大到更小, 需要上浮
22.   end if
23. end for
24. end for
    
```

顶点 i	1	2	3	4	5	6
$\lambda[i]$		1 √入	12入	∞	∞	∞
$\lambda[i]$			↓10浮	↓ 4 √入	∞	∞
$\lambda[i]$			↓ 8 √浮		↓17入	↓19入
$\lambda[i]$					↓ 13 √浮	19
$\lambda[i]$						↓ 17 √浮
$\lambda[i]$		1 √	8 √	4 √	13 √	17 √



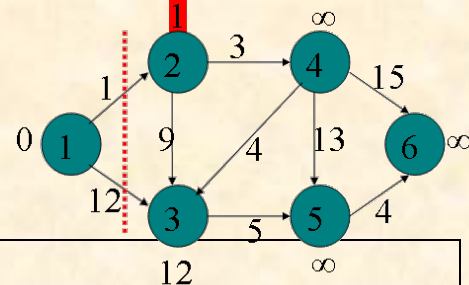
顶点i	1	2	3	4	5	6
$\lambda[i]$		$1\checkmark/\lambda$	12λ	∞	∞	∞
$\lambda[i]$			$\downarrow 10\text{浮}$	$\downarrow 4\checkmark/\lambda$	∞	∞
$\lambda[i]$			$\downarrow 8\checkmark/\text{浮}$		$\downarrow 17\lambda$	$\downarrow 19\lambda$
$\lambda[i]$					$\downarrow 13\checkmark/\text{浮}$	19
$\lambda[i]$						$\downarrow 17\checkmark/\text{浮}$
$\lambda[i]$		$1\checkmark$	$8\checkmark$	$4\checkmark$	$13\checkmark$	$17\checkmark$



顶点i	1	2	3	4	5	6
$\lambda[i]$		1✓入	12入	∞	∞	∞
$\lambda[i]$			↓10浮	↓4✓入	∞	∞
$\lambda[i]$			↓8✓浮		↓17入	↓19入
$\lambda[i]$					↓13✓浮	19
$\lambda[i]$						↓17✓浮
$\lambda[i]$		1✓	8✓	4✓	13✓	17✓



最短路径算法的堆优化：主要进程



```

12. for j ← 1 to n-1
13.   y ← DELETMIN(H) //每次挑中一个顶点，共执行n-1次
14.   Y ← Y - {y}, X ← X ∪ {y},
15.   for 每个邻接于y的顶点 w ∈ Y //邻接表，次数 < m
16.     if  $\lambda[y] + \text{length}[y, w] < \lambda[w]$  then //与边数有关，次数 < m
17.        $\lambda[w] \leftarrow \lambda[y] + \text{length}[y, w]$  //次数 < 第16步的执行次数 < m
18.       key(w) ←  $\lambda[w]$ 
19.     end if
20.     if w !∈ H then INSERT(H, w) //最少0次，全部次数 = n-1
21.     else SIFT-UP(H, H-1(w)) //最多m-(n-1)次，每次O(logn)
22.   end if
23. end for
24. end for
    
```

20, 21步，跟顶点的入度有关。图的总入度为m，其中：

n-1个顶点，每个入堆1次且仅1次，对应于n-1个入度，贡献给了插入堆操作；
剩下的m-(n-1)个入度，贡献给了堆上浮操作（至多）。



最短路径算法的堆优化：主要进程

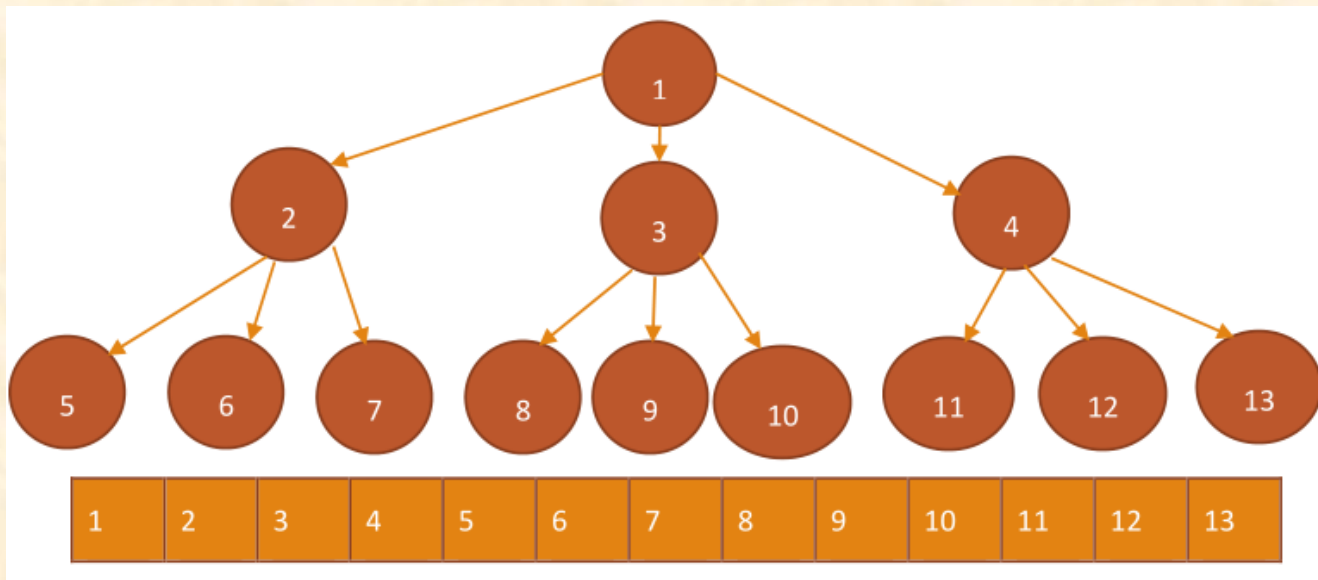
```
12. for j ← 1 to n-1
13.   y ← DELETETEMIN(H) //每次挑中一个顶点，共执行n-1次
14.   Y ← Y - {y}, X ← X ∪ {y},
15.   for 每个邻接于y的顶点 w ∈ Y //邻接表，次数 < m
16.     if  $\lambda[y] + \text{length}[y, w] < \lambda[w]$  then //与边数有关，次数 < m
17.        $\lambda[w] \leftarrow \lambda[y] + \text{length}[y, w]$  //次数 < 第16步的执行次数 < m
18.       key(w) ←  $\lambda[w]$ 
19.     end if
20.     if w ∉ H then INSERT(H, w) //最少0次，全部次数 = n-1
21.     else SIFT-UP(H, H-1(w)) //图示，最多m-(n-1)次，每次O(logn)
22.   end if
23. end for
24. end for
```

运算次数小结：恰好n-1次删除堆顶，恰好n-1次插入堆，最多m-n+1次堆上浮，每次操作都是O(logn)的，共需时间(m+2n-3)*O(logn)=O(m logn)，当m ≥ n^{1+ε} (>n logn)，即稠密图时成立。可记为O(|E| log |V|)。



最短路径算法的堆优化：进一步的研究

- 每次堆操作的时间是 $O(\log n)$ ，这个上界太松弛，仍然可以进一步压缩，使上界更紧致，待研究...
- 可以将二分堆推广为 d 堆(每个父亲最多 d 个儿子)，运行时间为 $O(\mathbf{nd} \cdot \log_d n + \mathbf{m} \cdot \log_d n)$





关于d-堆和d-堆排序（来自学生的初步研究）

1) Sift down:

$$T(n) < d * \log_d n$$

$$T(n) = O\left(\frac{d * \log n}{\log d}\right)$$

2) Makeheap:

$$T(n) = O\left(\frac{n}{d} * \frac{d * \log n}{\log d}\right) = O\left(\frac{n * \log n}{\log d}\right)$$

3) Heapsort:

$$T(n) = O\left(\frac{n * d * \log n}{\log d}\right)$$



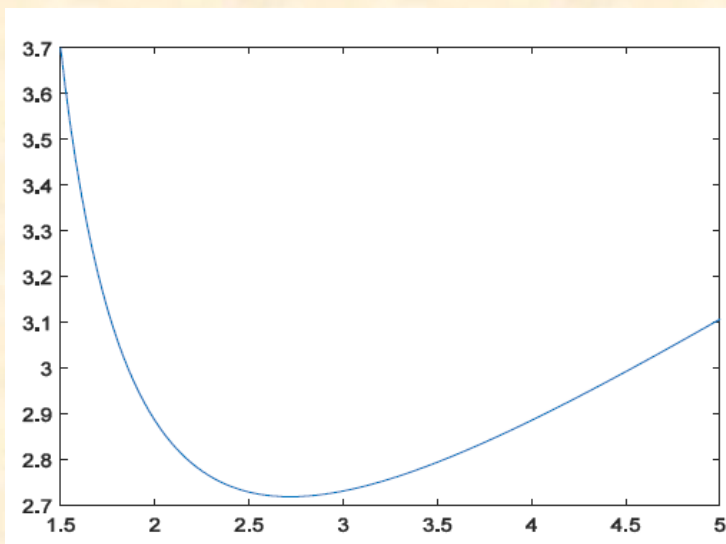
南京理工大学

关于d-堆和d-堆排序（来自学生的初步研究）

如上所示，d 叉堆中 d 的引入导致时间复杂度的计算公式由原来的 $n * \log n$ 变化为

$\frac{n*d*\log n}{\log d}$ 其中受影响的只有一个参数，就是 $d/\log n$ ，关于 $d/\log n$ 的增减性，可以对其进行

行求导可得当 $d=e$ 时，时间取得极小，图像如下：



$$y=x/\log x \quad x \in (1.5,5)$$



最短路径算法的堆优化：进一步的研究

- 每次堆操作的时间是 $O(\log n)$ ，这个上界太松弛，仍然可以进一步压缩，使上界更紧致，待研究...
- 可以将二分堆推广为 d 堆(每个父亲最多 d 个儿子)，运行时间为 $O(\textcolor{red}{n}d \cdot \log_d n + \textcolor{red}{m} \cdot \log_d n)$

	操作次数	每次操作耗时	小计耗时
插入堆	正好 $n-1$	$O(\log_d n)$	$O(\textcolor{red}{n} \log_d n)$
删除堆顶	正好 $n-1$	$O(\textcolor{red}{d} \log_d n)$	$O(\textcolor{red}{n} d \log_d n)$
堆上浮	最多 $m-(n-1)$	$O(\log_d n)$	$O(\textcolor{red}{m} \log_d n + \textcolor{red}{n} \log_d n)$
总计		$O(\textcolor{red}{n} d \log_d n + \textcolor{red}{m} \log_d n)$	



最短路径算法的堆优化：进一步的研究

- 每次堆操作的时间是 $O(\log n)$ ，这个上界太松弛，仍然可以进一步压缩，使上界更紧致，待研究...
- 可以将二分堆推广为 d 堆(每个父亲最多 d 个儿子)，运行时间为 $O(\textcolor{red}{nd} * \log_d n + \textcolor{red}{m} * \log_d n)$
- 选择合适的 $d = \lceil 2 + m/n \rceil$ ，可证明时间上界是 $O(m \log_{\lceil 2 + m/n \rceil} n)$
- 若图是稠密的，即 $m \geq n^{1+\epsilon}$ ，其中 $\epsilon > 0$ ，则可进一步证明，时间上界是 $O(\frac{m}{\epsilon})$



教材勘误

pp.134

算法 8.2 SHORTESTPATH

输入：含权有向图 $G=(V, E)$, $V=\{1, 2, \dots, n\}$ 。

输出： G 中顶点 1 到其他顶点的距离。假设已有一个空堆 H 。

```

1.  $Y \leftarrow V - \{1\}$ ;  $\lambda[1] \leftarrow 0$ ;  $key(1) \leftarrow \lambda[1]$ 
2. for  $y \leftarrow 2$  to  $n$ 
3.   if  $y$  邻接于 1 then
4.      $\lambda[y] \leftarrow length[1, y]$ 
5.      $key(y) \leftarrow \lambda[y]$ 
6.     INSERT( $H, y$ )
7.   else
8.      $\lambda[y] \leftarrow \infty$ 
9.      $key(y) \leftarrow \lambda[y]$ 
10.  end if
11. end for
12. for  $j \leftarrow 1$  to  $n-1$ 
13.    $y \leftarrow DELETEMIN(H)$ 
14.    $Y \leftarrow Y - \{y\}$  |将顶点  $y$  从  $Y$  删除|
15.   for 每个邻接于  $y$  的顶点  $w \in Y$ 
16.     if  $\lambda[y] + length[y, w] < \lambda[w]$  then
17.        $\lambda[w] \leftarrow \lambda[y] + length[y, w]$ 
18.        $key(w) \leftarrow \lambda[w]$ 
19.     end if
20.     if  $w \notin H$  then INSERT( $H, w$ )
21.     else SIFTUP( $H, H^{-1}(w)$ )
22.   end if
23. end for
24. end for
    
```

需要强调的是输入算法的是邻接表，如果用邻接矩阵，我们最终会得到一个 $\Theta(n^2)$ 时间的算法。分配给每个顶点 $y \in Y$ 一个键，它是连接 1 到 y 的边的耗费（如果边存在的话），否则键就被置成 ∞ 。堆 H 一开始包含了与顶点 1 相邻接的所有顶点，第 13 步中 for 循环的每一次迭代从选出带有最小键的顶点 y 开始，然后更新 Y 中与 y 邻接的每一个顶点 w 的键，接着，如果 w 不在堆中，则插入；否则，如果必要就上渗。函数 $H^{-1}(w)$ 返回 w 在 H 中的位置，这仅仅用一个数组就可以直接来实现，数组的第 j 项是顶点 j 在堆中的位置（注意堆由一个数组 $H[1 \dots n]$ 来实现）。运行时间主要取决于堆运算，这里共有 $n-1$ 个 DELETEMIN 运算， $n-1$ 个 INSERT 运算，最多 $m-n+1$ 个 SIFTUP 运算，每个堆运算用 $O(\log n)$ 时间，得到总共需要 $O(m \log n)$ 时间。

可以回想一下， d 堆本质上是二堆的推广，它的树中的每一个内部节点最多有 d 个孩子而不是 2 个，这里 d 是一个可以任意大的数（见练习 4.21）。如果我们使用 d 堆，运行时



教材勘误

数组 $H[1 \cdots n]$ 来实现)。运行时间主要取决于堆运算，这里共有 $n - 1$ 个 DELETETMIN 运算， $n - 1$ 个 INSERT 运算，最多 $m - n + 1$ 个 SIFTUP 运算，每个堆运算用 $O(\log n)$ 时间，得到总共需要 $O(m \log n)$ 时间。

可以回想一下， d 堆本质上是二分堆的推广，它的树中的每一个内部节点最多有 d 个孩子而不是 2 个，这里 d 是一个可以任意大的数（见练习 4.21）。如果我们使用 d 堆，运行时

准确地说，是 $O(m \log n + n \log n)$

数组 $H[1 \cdots n]$ 来实现)。运行时间主要取决于堆运算，这里共有 $n - 1$ 个 DELETETMIN 运算， $n - 1$ 个 INSERT 运算，最多 $m - n + 1$ 个 SIFTUP 运算，每个堆运算用 $O(\log n)$ 时间，得到总共需要 $O(m \log n)$ 时间。

可以回想一下， d 堆本质上是二分堆的推广，它的树中的每一个内部节点最多有 d 个孩子而不是 2 个，这里 d 是一个可以任意大的数（见练习 4.21）。如果我们使用 d 堆，运行时



最小生成树

- 设 $G=(V,E)$ 是连通无向带权图, (V,T) 是 G 的一个子图, 并且 T 是一颗树, 那么称 (V,T) 是 G 的生成树。如果 T 的权之和是所有生成树中最小的, 那么则称之为最小生成树。
- 网络的最小生成树在实际中有广泛应用。例如, 在设计通信网络时, 用图的顶点表示城市, 用边 (v,w) 的权 $c[v][w]$ 表示建立城市 v 和城市 w 之间的通信线路所需的费用, 则最小生成树就给出了建立通信网络的最经济的方案。
- 假定 G 是连通的, 如果 G 是非连通的, 那么, 可以对 G 的每个子图应用求解最小生成树的算法。

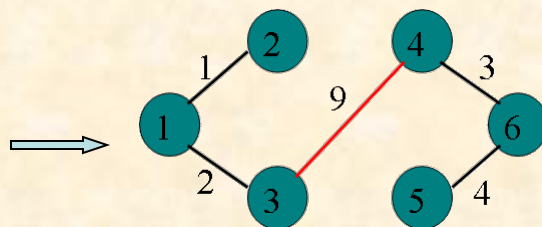
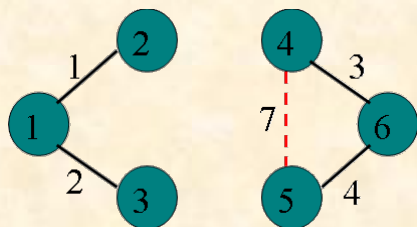
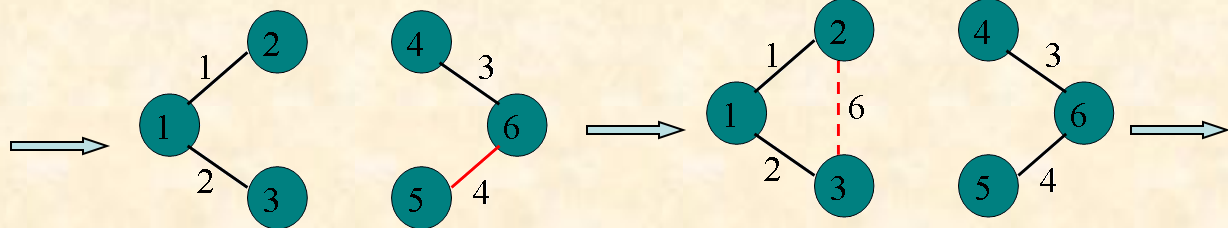
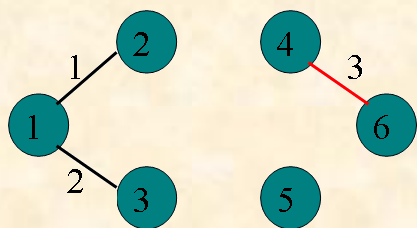
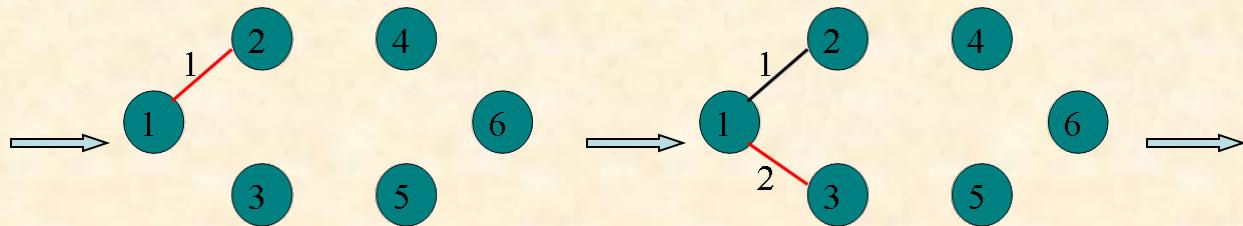
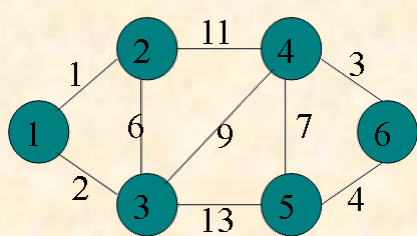


Kruskal算法

1. 对 G 的边 E 按权重以非降序排列。
2. 初始时输出树 $T=\{\}$ ；依次取排序表中的每条边，若加入 T 不形成回路，则加入 T ；否则将其丢弃；
3. 不断重复步骤2，直到树 T 包含 $n-1$ 条边，算法结束。



南京理工大学





Kruskal算法

1. 对E中的边按权重以非降序排列
2. for 每个顶点 $v \in V$
3. MAKESET($\{v\}$) //why?
4. end for
5. $T = \{\}$
6. while $|T| < n-1$
7. 令(x,y)为E中的下一条边
8. if FIND(x) $\neq$ FIND(y) then
9. 将(x,y)加入T
10. UNION(x,y)
11. end if
12. end while

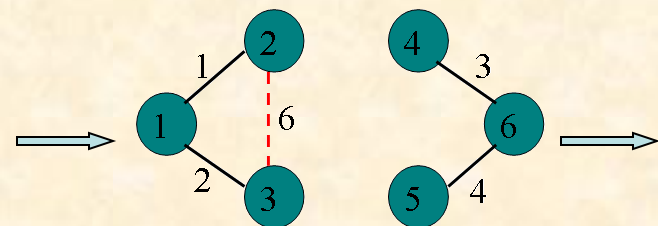
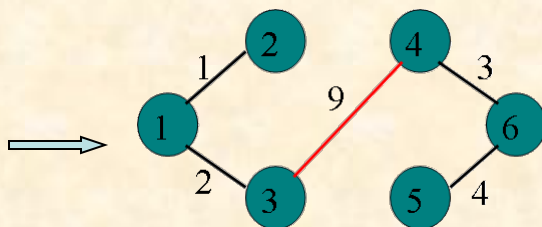
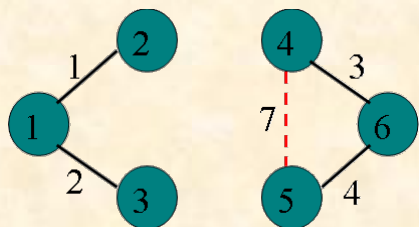
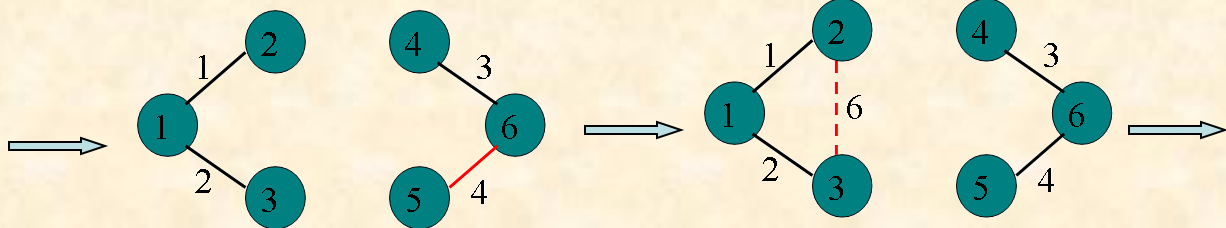
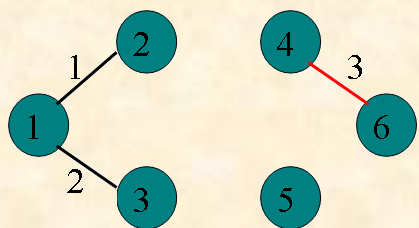
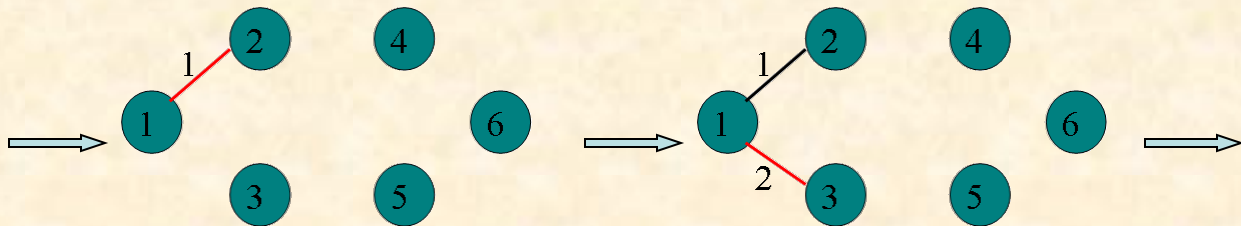
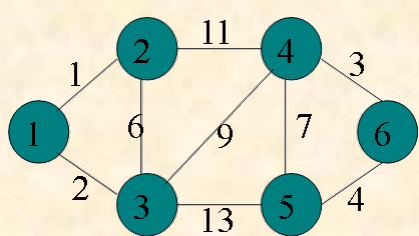


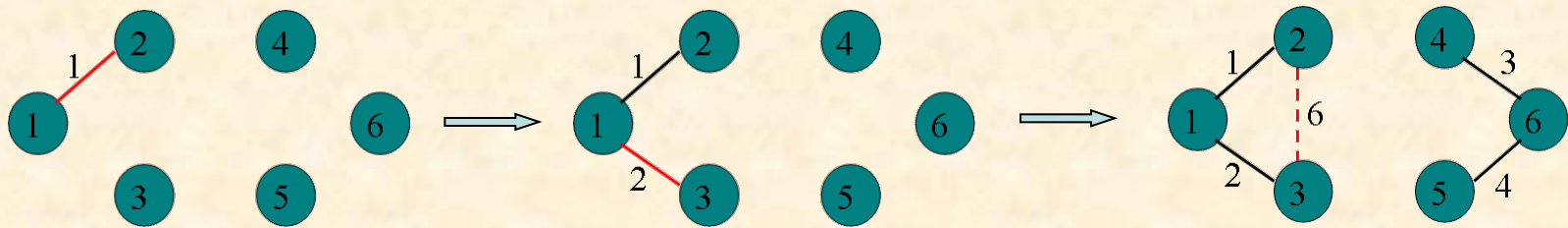
Kruskal算法

1. 对E中的边按权重以非降序排列 **//O(mlogm)**
2. for 每个顶点 $v \in V$ **//O(n)**
3. MAKESET($\{v\}$) //why?
4. end for
5. $T = \{\}$
6. while $|T| < n-1$
7. 令(x,y)为E中的下一条边 **//试探每条边, 最多m次, 至少n-1次**
8. if FIND(x) $\neq$ FIND(y) then
9. 将(x,y)加入T
10. UNION(x,y)
11. end if
12. end while

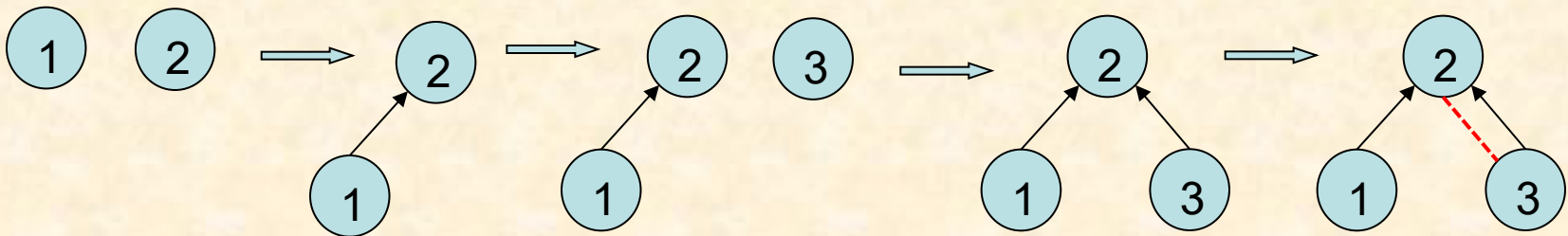


南京理工大学





8. if $\text{FIND}(x) \neq \text{FIND}(y)$ then //最多 $2m$ 次查找, 原因如上
9. 将 (x,y) 加入T //刚好 $n-1$ 次, $\Theta(n)$
10. $\text{UNION}(x,y)$ //如上, 刚好 $n-1$ 次, $\Theta(n)$





Kruskal算法

1. 对E中的边按权重以非降序排列 **//O(mlogm)**
2. for 每个顶点 $v \in V$ **// $\Theta(n)$**
3. MAKESET($\{v\}$) **//why?**
4. end for
5. $T = \{\}$
6. while $|T| < n-1$
7. 令(x,y)为E中的下一条边 **//试探每条边, 最多m次, 至少n-1次**
8. if FIND(x) $\neq$ FIND(y) then **//最多2m次查找, 原因如上**
9. 将(x,y)加入T **//刚好n-1次, $\Theta(n)$**
10. UNION(x,y) **//如上, 刚好n-1次, $\Theta(n)$**
11. end if
12. end while

$$O(m \log m) + \Theta(n) + O(m) + O(m) + \Theta(n) + \Theta(n) = O(m \log m + n)$$



正确性证明

我们只要证明，使用Kruskal 算法过程中，每次循环所得到的 T (从空集增至最小生成树)总是图 G 的最小生成树的子集即可。

证明使用“归纳法+反证法”。

- (1) G 总是具有一个最小生成树，不妨记为 T^*
- (2) 当前要加入的边为 $e = (x, y)$
- (3) 包含 x 的那棵子树的所有顶点用 X 表示，有 $x \in X, y \in V - X$
- (4) 假设在 $e = (x, y)$ 加入之前得到的 T 均满足 $T \subset T^*$



令 $T' = T \cup \{e\}$ ，下面我们要证明 $T' \subset T^*$ 。

依据归纳假设，有 $T \subset T^*$ 。

i) 如果 $e \in T^*$ ，显然有 $T' \subset T^*$

ii) 如果 $e \notin T^*$ ，我们知道 T^* 中必定包含这样一条边 $e' = (w, z)$ ，其中 $w \in X$ ， $z \in V - X$ ，且满足

$$\text{cost}(e') \geq \text{cost}(e),$$

否则，若 $\text{cost}(e') < \text{cost}(e)$ ，则 e' (而不是 e) 将被选择加入。

下面我们来研究：如果 $e' \in T^*$ 会带来什么结果。



定义 $T^{**} = T^* - \{e'\} \cup \{e\}$,

那么 T^{**} 也是图的一个生成树, 并且

$$\begin{aligned} \text{cost}(T^{**}) &= \text{cost}(T^*) - \text{cost}(e') + \text{cost}(e) \\ &< \text{cost}(T^*). \end{aligned}$$

也就是说, T^* 不是最小生成树, 矛盾。

所以 e 必定是属于 T^* 的。也就必定有 $T' \subset T^*$, 证毕。



Prim算法

- 设 $G=(V,E)$ 是连通无向带权图， $V=\{1,2,\dots,n\}$ 。构造 G 的最小生成树的Prim算法的基本思想是：
 - 首先置 $X=\{1\}$,
 - 然后，只要 X 是 V 的真子集，就作如下的贪心选择：
 - 选取权重最小的边 (x,y) , 其中 $x\in X$, $y\in Y$,
 - 将边 (x,y) 加入当前的最小生成树,
 - 将顶点 y 从 Y 移到 X 中,
 - 这个过程一直进行到 $X=V$ 时为止。
- 在这个过程中选取到的所有边恰好构成 G 的一棵最小生成树。



算法设计要点:

寻找 (x,y) , 使得 $x \in X$, $y \in Y$, 且 $\text{cost}(x,y)$ 最小.

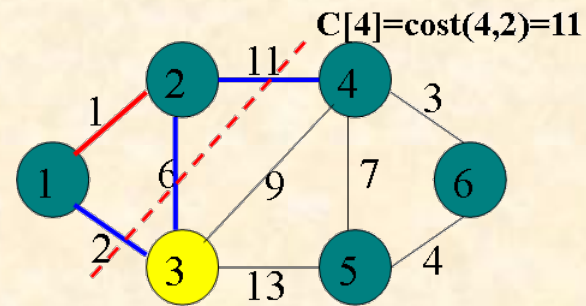
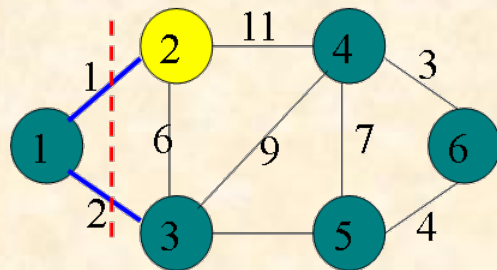
候选边 (x,y) 对应的顶点 $y \in Y$, 可考察 y 在 X 中的邻居.

邻居就是 X 中离 y 最近的那个点 x^* , 即

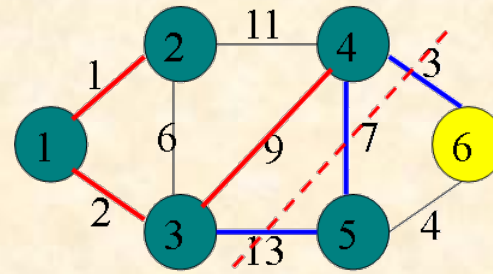
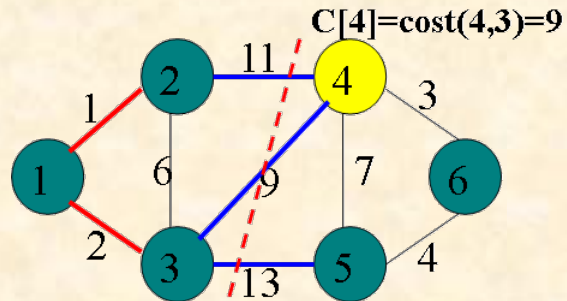
$\text{cost}(x^*, y) = \min \{ \text{cost}(x, y) \mid x \in X \text{ 且 } (x,y) \text{ 存在} \}$

记 $x^* = N[y]$, 定义 $C[y] = \text{cost}(x^*, y) = \text{cost}(y, N[y])$.

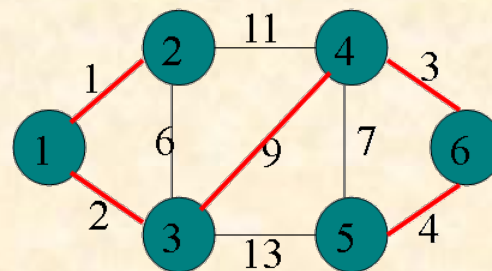
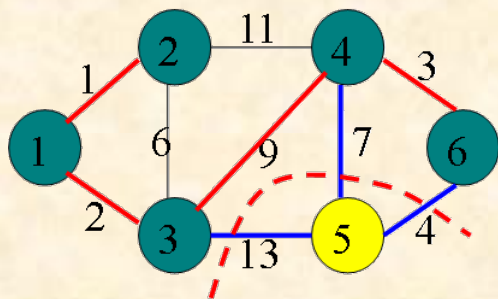
X, Y 可用 n 维 $\{0,1\}$ 向量表示, 例 $X[i]=1$ 表示顶点 $i \in X$.



	v1	v2	v3	v4	v5	v6	X
(vtex, cost)		$\checkmark(v1, \mathbf{1})$	(v1,2)				{v1}
(vtex, cost)			$\checkmark(v1, \mathbf{2})$	(v2,11)			{v1,v2}
(vtex, cost)							{v1,v2,v3}



	v1	v2	v3	v4	v5	v6	X
(vtex, cost)		$\checkmark(v1, 1)$	(v1, 2)				v1
(vtex, cost)			$\checkmark(v1, 2)$	(v2, 11)			v1v2
(vtex, cost)				$\checkmark(v3, 9) \downarrow$	(v3, 13)		v1v2v3
(vtex, cost)					(v4, 7) $\downarrow$	$\checkmark(v4, 3)$	v1v2v3v4
(vtex, cost)							v1v2v3v4v6



	v1	v2	v3	v4	v5	v6	X
(vtex,cost)		✓(v1,1)	(v1,2)				v1
			✓(v1,2)	(v2,11)			v1v2
				✓(v3,9)↓	(v3,13)		v1v2v3
					(v4,7) ↓	✓(v4,3)	v1v2v3v4
					✓(v6,4) ↓		v1v2v3v4v6
		✓	✓	✓	✓	✓	v1~v6



```
1.  $T \leftarrow \{\}$ ;  $X \leftarrow \{1\}$ ;  $Y \leftarrow V - \{1\}$     //  $\Theta(n)$ 
2. for  $y \leftarrow 2$  to  $n$ 
3.   if  $y$  邻接于  $1$  then
4.      $N[y] \leftarrow 1$ 
5.      $C[y] \leftarrow \text{cost}[1, y]$ 
6.   else  $C[y] \leftarrow \infty$ 
7.   end if
8. end for
9. for  $j \leftarrow 1$  to  $n-1$  //寻找MST的 $n-1$ 条边
10.  令  $y \in Y$ , 使得  $C[y]$  最小
11.   $T \leftarrow T \cup (y, N[y])$ 
12.   $X \leftarrow X \cup \{y\}$ 
13.   $Y \leftarrow Y - \{y\}$ 
14.  for 每个邻接于  $y$  的顶点  $w \in Y$ 
15.    if  $\text{cost}[y, w] < C[w]$  then
16.       $N[w] \leftarrow y$ 
17.       $C[w] \leftarrow \text{cost}[y, w]$ 
18.    end if
19.  end for
20. end for
```



```
1. T ← {}; X ← {1}; Y ← V - {1}    //Θ(n)
2. for y ← 2 to n                    //Θ(n)
3.   if y 邻接于 1 then                //第3~6步, O(n)
4.     N[y] ← 1
5.     C[y] ← cost[1, y]
6.   else C[y] ← ∞
7.   end if
8. end for
9. for j ← 1 to n-1 //寻找MST的n-1条边
10.  令 y ∈ Y, 使得C[y]最小 //每次Θ(n), 共n-1次, 计Θ(n²)
11.  T ← T ∪ (y, N[y]) //Θ(1) × (n-1) = Θ(n)
12.  X ← X ∪ {y}       //Θ(1) × (n-1) = Θ(n)
13.  Y ← Y - {y}       //...
14.  for 每个邻接于y的顶点w ∈ Y //邻接表, 每条边检查2次, 共2m次, Θ(m)
15.    if cost[y, w] < C[w] then //确定w ∈ Y时, 每条边检查1次, 共m次
16.      N[w] ← y //少于第15步的次数, 最多m次, Θ(m)
17.      C[w] ← cost[y, w] //Θ(m)
18.    end if
19.  end for
20. end for
```

$$\Theta(n) + \Theta(n^2) + \Theta(m) = \Theta(m + n^2)$$

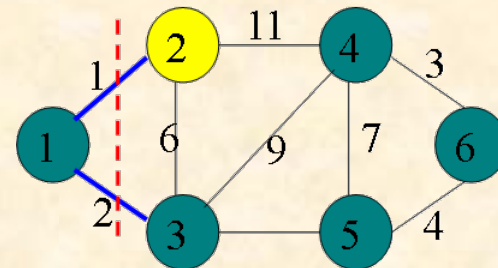


Prim算法的正确性

- 证明：略。(pp.139-140)



Prim算法的堆优化

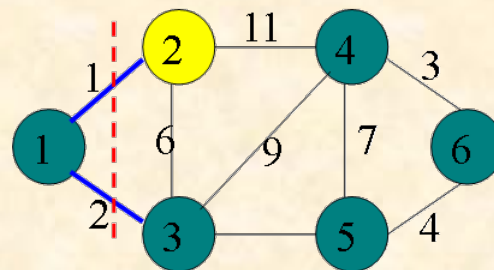


9. for $j \leftarrow 1$ to $n-1$ //寻找MST的 $n-1$ 条边
10. 令 $y \in Y$, 使得 $C[y]$ 最小 //每次 $\Theta(n)$, 共 $n-1$ 次, 计 $\Theta(n^2)$
11. $T \leftarrow T \cup (y, N[y])$ // $\Theta(1) \times (n-1) = \Theta(n)$
12. $X \leftarrow X \cup \{y\}$ // $\Theta(1) \times (n-1) = \Theta(n)$
13. $Y \leftarrow Y - \{y\}$ //...
14. for 每个邻接于 y 的顶点 $w \in Y$ //邻接表, 每条边检查 2 次, 共 2m 次, $\Theta(m)$
15. if $\text{cost}[y, w] < C[w]$ then //确定 $w \in Y$ 时, 每条边检查 1 次, 共 m 次
16. $N[w] \leftarrow y$ //少于第15步的次数, 最多 m 次, $\Theta(m)$
17. $C[w] \leftarrow \text{cost}[y, w]$ // $\Theta(m)$
18. end if
19. end for
20. end for

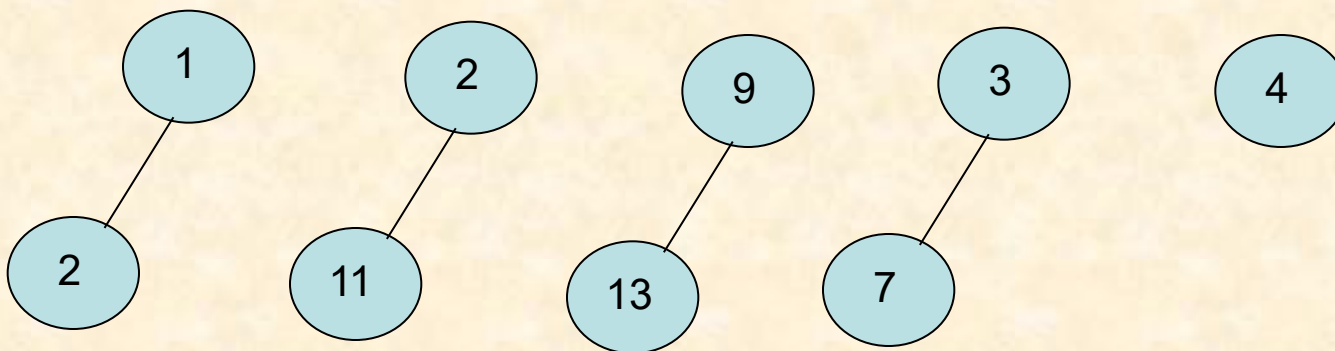
v1	v2	v3	v4	v5	v6
	$\checkmark(v1, 1)$	$(v1, 2)$			
		$\checkmark(v1, 2)$	$(v2, 11)$		
			$\checkmark(v3, 2) \downarrow$	$(v3, 13)$	
				$(v4, 7) \downarrow$	$\checkmark(v4, 3)$
				$\checkmark(v6, 4) \downarrow$	



Prim算法的堆优化

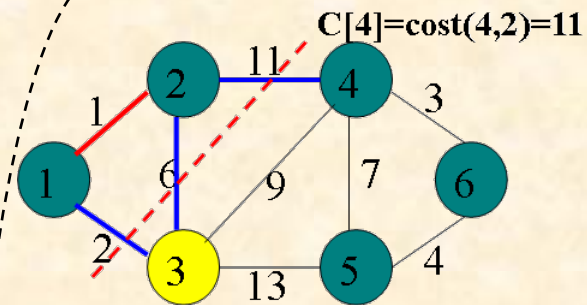
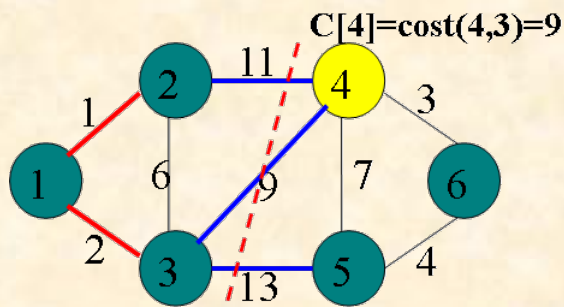
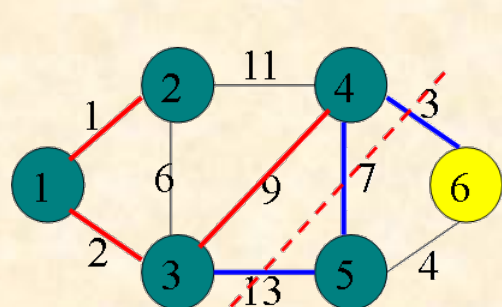
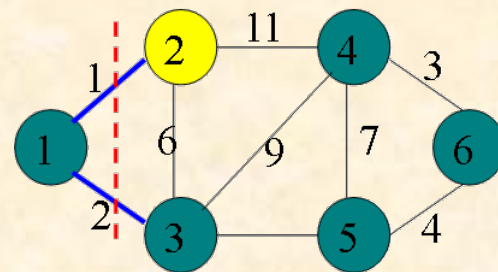


v1	v2	v3	v4	v5	v6
	$\checkmark(v1,1)\lambda$	$(v1,2)\lambda$			
		$\checkmark(v1,2)$	$(v2,11)\lambda$		
			$\checkmark(v3,9)\downarrow\text{浮}$	$(v3,13)\lambda$	
				$(v4,7)\downarrow\text{浮}$	$\checkmark(v4,3)\lambda$
				$\checkmark(v6,4)\downarrow\text{浮}$	
	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$





Prim算法的堆优化



v1	v2	v3	v4	v5	v6
	$\checkmark(v1,1)\lambda$	$(v1,2)\lambda$			
		$\checkmark(v1,2)$	$(v2,11)\lambda$		
			$\checkmark(v3,9)\downarrow\text{浮}$	$(v3,13)\lambda$	
				$(v4,7)\downarrow\text{浮}$	$\checkmark(v4,3)\lambda$
				$\checkmark(v6,4)\downarrow\text{浮}$	
	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$



Prim算法的堆优化

- 类似于Dijkstra最短路径算法的堆优化，略。
- 复杂度分析：类似，略。
- 可推广为用d堆的情况，类似，略。

	操作次数	每次操作耗时	小计耗时
插入堆	正好 $n-1$	$O(\log n)$	$O(n \log n)$
删除堆顶	正好 $n-1$	$O(\log n)$	$O(n \log n)$
堆上浮	最多 $m-(n-1)$	$O(\log n)$	$O(m \log n)$
总计			$O(m \log n)$



Prim算法的优化, you need know more...

Table
Mini

Algo

Boru
Krus

Krus

Prim

Prim
Prim
Prim

Yao
Cher

Fred
Gabc

Karg
Karg

^a $\alpha(n)$
^b $\beta(n)$
^cExl



PERGAMON

Computers & Operations Research 28 (2001) 767–785

computers &
operations
research

www.elsevier.com/locate/dsw

Minimum-weight spanning tree algorithms A survey and empirical study

Cüneyt F. Bazlamaççı^a, Khalil S. Hindi^{b,*}

^a*Electrical and Electronics Engineering Department, Middle East Technical University, Ankara 06531, Turkey*

^b*Department of Systems Engineering, Brunel University, Uxbridge, Middlesex UB8 3PH, UK*

Received 1 August 1999; received in revised form 1 December 1999

Abstract

The minimum-weight spanning tree problem is one of the most typical and well-known problems of combinatorial optimisation. Efficient solution techniques had been known for many years. However, in the last two decades asymptotically faster algorithms have been invented. Each new algorithm brought the time bound one step closer to linearity and finally Karger, Klein and Tarjan proposed the only known expected linear-time method. Modern algorithms make use of more advanced data structures and appear to be more complicated to implement. Most authors and practitioners refer to these but still use the classical ones, which are considerably simpler but asymptotically slower. The paper first presents a survey of the classical methods and the more recent algorithmic developments. Modern algorithms are then compared with the classical ones and their relative performance is evaluated through extensive empirical tests, using reasonably large-size problem instances. Randomly generated problem instances used in the tests range from small networks having 512 nodes and 1024 edges to quite large ones with 16384 nodes and 524288 edges. The purpose of the comparative study is to investigate the conjecture that modern algorithms are also easy to apply and have constants of proportionality small enough to make them competitive in practice with the older ones.

$n)$



姚期智：图灵奖（Turing Award）

1946年12月24日生，

美籍华人，计算机科学家，

2000年图灵奖得主，

目前唯一获得此奖项的华人及亚洲人。

目前是清华大学**理论计算机科学研究中心**教授。





姚期智:开华裔获图灵之先河

姚期智进入计算机科学领域最早的论文之一《寻找最小生成树的 $O(|E| \log \log |V|)$ 算法》一文就引起轰动，因为学术界原先认为，寻找最小生成树算法的时间复杂度的下界是 $O(|E| \log |V|)$ ，而姚期智的论文证明这个极限是可以打破的（时年29岁）。

$$O(|E| \log |V|) \longrightarrow O(|E| \log \log |V|)$$



$$O(|E| \log |V|) \longrightarrow O(|E| \log \log |V|)$$

Yao, A. C. (1975) "An $O(|E| \log \log |V|)$ algorithm for finding minimum spanning trees", *Information Processing Letters*, 4(1), 21–23.



Information Processing Letters

Volume 4, Issue 1, September 1975, Pages 21–23



Volume 4, number 1

INFORMATION PROCESSING LETTERS

September 1975

AN $O(|E| \log \log |V|)$ ALGORITHM FOR FINDING MINIMUM SPANNING TREES *

Andrew Chi-chih YAO

*Department of Computer Science, University of Illinois,
Urbana, Illinois 61801, USA*

Received 30 December 1975, revised version received 9 June 1975

Minimum spanning tree, linear median finding algorithm

dent with all the nodes. Therefore, the total number of operations required by our algorithm is

$$O\left(|E| \log k + \frac{|E|}{k} \log |V|\right),$$

which is $O(|E| \log \log |V|)$ if we choose k to be $\log |V|$.



把 $\log n$ 优化为 $\log \log n$ ，有什么用？

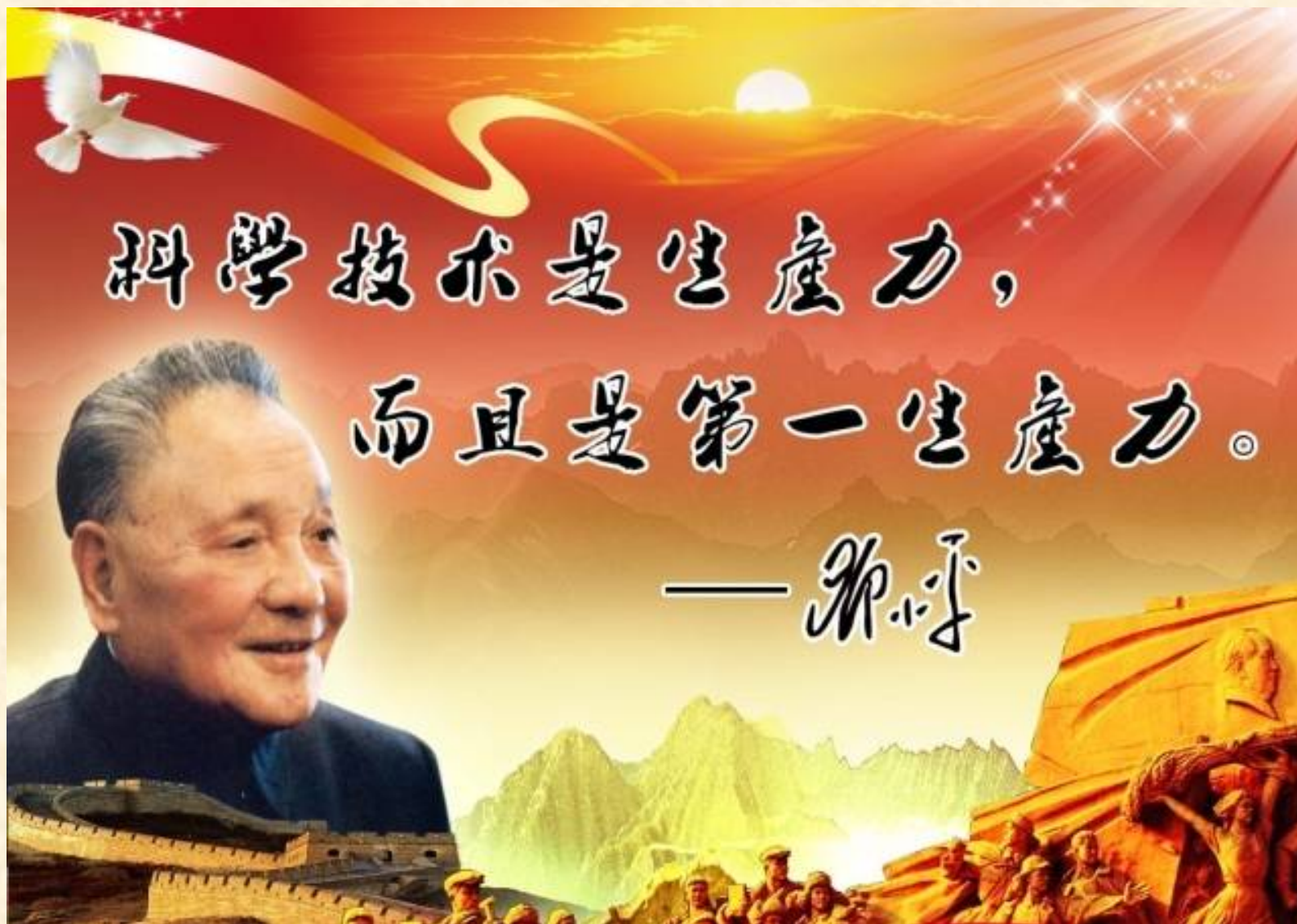
我们不妨做一个简单的近似，假设全宇宙都是氢，宇宙中有1000亿星系，每个星系有1000亿恒星，每颗恒星都是一倍太阳质量，这样算出来宇宙中重子物质的总质量（不包括质量更大的暗物质，也不考虑星际介质和MACHO等不可见重子物质）是 $M_{\text{Cosmos}} = 2 \times 10^{52} \text{ kg}$ 。再考虑氢原子的质量，大约是 $m_{\text{H}} = 1.67 \times 10^{-27} \text{ kg}$ ，一除就可以得到，宇宙中大致有 10^{79} 个原子。

宇宙中原子数目 $n = 10^{78 \sim 82}$ ，不妨取 $n = 10^{80} = 2^{265.75}$

n	10^{80}	Ref.
n^2	10^{160}	Prim(1957)
$\log n$	265.75	Prim Heap
$\log \log n$	8.05	Yao(1975)



南京理工大学





止於至善